Repository Verbatim Extraction

Repository: SlackLMS
Branch: main
Generated on: 2026-03-25

File Inventory

A. Docs folder CSV schema files / schema references
docs/csv-sheetdb-schema.md
docs/generated_csv/audit_log.csv
docs/generated_csv/audit_logs.csv
docs/generated_csv/automations.csv
docs/generated_csv/delivery_queue.csv
docs/generated_csv/enrollment.csv
docs/generated_csv/enrollments.csv
docs/generated_csv/learner_progress.csv
docs/generated_csv/learners.csv
docs/generated_csv/lessons.csv
docs/generated_csv/retry_queue.csv
docs/generated_csv/students.csv
docs/generated_csv/submission_log.csv

B. .gs files on main branch
00_WebAppEntry.gs
01_SlackRouter.gs
02_SlackPayloadParser.gs
03_SlackService.gs
04_SlackSecurity.gs
05_LmsEnrollmentService.gs
06_LmsLessonService.gs
07_LmsProgressService.gs
08_LmsCompletionService.gs
09_LmsReminderService.gs
10_LmsReportService.gs
11_SlackApiClient.gs
12_SlackBlockKitBuilder.gs
13_LearnerProgressStateMachine.gs
14_Scheduler.gs
15_SheetsDataSync.gs
16_ConfigBootstrap.gs
17_HealthMonitor.gs
18_RetryResolver.gs
19_HostTestHarness.gs
20_Errors.gs
21_Util.gs
22_ScriptProperties.gs
23_Config.gs
24_SheetsGateway.gs
25_SchemaRegistry.gs
26_TableRepository.gs
27_TransactionManager.gs
28_AuditLogger.gs
29_DbClient.gs
30_SheetDb.gs
code.gs
sheet_db_lib/00_SheetDb.gs
sheet_db_lib/01_DbClient.gs
sheet_db_lib/02_Config.gs
sheet_db_lib/03_Errors.gs
sheet_db_lib/04_Util.gs
sheet_db_lib/05_SchemaRegistry.gs
sheet_db_lib/06_SheetsGateway.gs
sheet_db_lib/07_TableRepository.gs
sheet_db_lib/08_TransactionManager.gs
sheet_db_lib/09_AuditLogger.gs
sheet_db_lib/10_ScriptProperties.gs
slack_host/00_WebAppEntry.gs
slack_host/01_SlackRouter.gs
slack_host/02_SlackPayloadParser.gs

```
slack_host/03_SlackService.gs
slack_host/04_SlackSecurity.gs
slack_host/05_LmsEnrollmentService.gs
slack_host/06_LmsAutomationService.gs
slack_host/07_RequestContextFactory.gs
slack_host/08_ResultFormatter.gs
slack_host/09_WorkflowWebhookHandlers.gs
slack_host/10_HostConfig.gs
slack_host/11_HostTestHarness.gs
```

FILE PATH: docs/csv-sheetdb-schema.md

RWR LMS CSV and SheetDb Schema Inventory

Section 1 â CSV File Inventory

CSV file scan result
- No physical ‘*.csv‘ files were originally present in the repository; generated header-only CSV a
rtifacts now live under ‘docs/generated_csv/‘.
- The codebase is sheet-backed: tab names are registered through ‘db.schema(...)‘ and materialized
 through ‘SheetsGateway.ensureTable(...)‘ / ‘SpreadsheetApp.getSheetByName(...)‘.
- No ‘headers.indexOf(...)‘-driven dynamic CSV parsing was found.
- No direct ‘sheet.getDataRange().getValues()‘ consumers outside the shared ‘SheetsGateway.readTab
le(...)‘ helper were found.

Section 2 â Sheet / Table Inventory

Discovery summary
Unique sheet-backed tables found across the repo:
- Main RWR LMS runtime: ‘learners‘, ‘enrollment‘, ‘lessons‘, ‘learner_progress‘, ‘submission_log‘,
 ‘delivery_queue‘, ‘retry_queue‘, ‘audit_log‘
- Split Slack host example app: ‘students‘, ‘enrollments‘, ‘automations‘, ‘audit_logs‘

Generated CSV artifacts
- Header-only CSV files were generated for every discovered table under ‘docs/generated_csv/‘ so t
he sheet/tab schemas can be copied directly into spreadsheet imports or seed workflows.

Table: ‘learners‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘learners‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘slackUserId‘
  3. ‘email‘
  4. ‘name‘
  5. ‘status‘
  6. ‘createdAt‘
  7. ‘updatedAt‘
  8. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, required at persistence level, primary key. Auto-generated with ‘Util.generateId
(’learners’)‘ when omitted.
  - ‘slackUserId‘: String, effectively required by application flow because lookups/enrollment dep
end on it.
  - ‘email‘: String, optional, defaults to empty string in ‘ensureLearnerRecord()‘.
  - ‘name‘: String, optional, defaults to empty string in ‘ensureLearnerRecord()‘.
  - ‘status‘: String, required by business flow on insert, observed enum values: ‘active‘.
  - ‘createdAt‘: DateTime, auto-populated on insert.
  - ‘updatedAt‘: DateTime, auto-populated on insert/update.
  - ‘deletedAt‘: DateTime, optional, soft-delete marker used by the repository filter.
- **Foreign keys:** none.
- **Soft delete:** ‘{ enabled: true, column: ’deletedAt’, archivedValue: ’<timestamp set on remove
>’ }‘
- **Observed active-record filter:** implicit in ‘TableRepository.findAll()‘ because rows with bla
nk ‘deletedAt‘ only are returned.
- **AppSheet consumer evidence:** none found.

Table: ‘enrollment‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘enrollment‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘

  2. ‘learnerId‘
  3. ‘courseId‘
  4. ‘track‘
  5. ‘status‘
  6. ‘createdAt‘
  7. ‘updatedAt‘
  8. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, required at persistence level, primary key. Auto-generated when omitted.
  - ‘learnerId‘: String, required by business flow, foreign key to ‘learners.id‘.
  - ‘courseId‘: String, required by business flow.
  - ‘track‘: String, optional in code path, defaults to configured track or ‘’ONBOARDING’’.
  - ‘status‘: String, required by business flow, observed enum values: ‘active‘.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** ‘learnerId -> learners.id‘.
- **Soft delete:** enabled via ‘deletedAt‘.
- **Observed active-record filter:** implicit repository ‘deletedAt‘ filter; reports also filter ‘status === ’active’’ for active enrollment counts.
- **AppSheet consumer evidence:** none found.

Table: ‘lessons‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘lessons‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘courseId‘
  3. ‘track‘
  4. ‘title‘
  5. ‘contentRef‘
  6. ‘active‘
  7. ‘createdAt‘
  8. ‘updatedAt‘
  9. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted, though runtime sync prefers caller-supplied ‘row.id‘.
  - ‘courseId‘: String, optional in sync helper, defaults to empty string.
  - ‘track‘: String, optional, defaults to empty string.
  - ‘title‘: String, optional, defaults to empty string.
  - ‘contentRef‘: String, optional, defaults to empty string.
  - ‘active‘: String/Boolean-like flag. Sync helper persists ‘String(...)‘; observed values: ‘’true’’, inherited row value.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** none enforced, though ‘courseId‘ behaves like a course reference.
- **Soft delete:** enabled via ‘deletedAt‘.
- **AppSheet consumer evidence:** none found.

Table: ‘learner_progress‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘learner_progress‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘learnerId‘
  3. ‘lessonId‘
  4. ‘state‘
  5. ‘dueAt‘
  6. ‘completedAt‘
  7. ‘createdAt‘

  8. ‘updatedAt‘
  9. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘learnerId‘: String, foreign key to ‘learners.id‘.
  - ‘lessonId‘: String, foreign key to ‘lessons.id‘.
  - ‘state‘: String, required by business logic once rows exist. Observed enum values from state m
achine and filters: ‘queued‘, ‘delivered‘, ‘submitted‘, ‘completed‘, ‘overdue‘.
  - ‘dueAt‘: DateTime, optional.
  - ‘completedAt‘: DateTime, optional.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated and also touched by state-machine logic.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** ‘learnerId -> learners.id‘, ‘lessonId -> lessons.id‘.
- **Soft delete:** enabled via ‘deletedAt‘.
- **Observed active-record filter:** implicit repository ‘deletedAt‘ filter; many flows additional
ly treat rows where ‘state !== ’completed’‘ as active lesson rows.
- **AppSheet consumer evidence:** none found.

Table: ‘submission_log‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘submission_log‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘learnerId‘
  3. ‘lessonId‘
  4. ‘submitKey‘
  5. ‘payload‘
  6. ‘createdAt‘
  7. ‘updatedAt‘
  8. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘learnerId‘: String, foreign key to ‘learners.id‘.
  - ‘lessonId‘: String, foreign key to ‘lessons.id‘.
  - ‘submitKey‘: String, dedupe/business key, computed as ‘learner.id + ’:’ + lessonId‘.
  - ‘payload‘: String, optional, defaults to empty string.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** ‘learnerId -> learners.id‘, ‘lessonId -> lessons.id‘.
- **Soft delete:** enabled via ‘deletedAt‘.
- **AppSheet consumer evidence:** none found.

Table: ‘delivery_queue‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘delivery_queue‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘learnerId‘
  3. ‘lessonId‘
  4. ‘status‘
  5. ‘runAt‘
  6. ‘createdAt‘
  7. ‘updatedAt‘
  8. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘learnerId‘: String, foreign key to ‘learners.id‘.
  - ‘lessonId‘: String, foreign key to ‘lessons.id‘, but intentionally blank in some queueing flow
s until next-lesson resolution is implemented.
  - ‘status‘: String, observed enum values: ‘queued‘, ‘delivered‘.

  - ‘runAt‘: DateTime, required by business flow on inserts; populated with ‘new Date().toISOStrin
g()‘.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** ‘learnerId -> learners.id‘, ‘lessonId -> lessons.id‘.
- **Soft delete:** enabled via ‘deletedAt‘.
- **Observed active-record filter:** implicit repository ‘deletedAt‘ filter; business processing a
dditionally filters ‘status === ’queued’‘.
- **AppSheet consumer evidence:** none found.

Table: ‘retry_queue‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘retry_queue‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘jobType‘
  3. ‘payload‘
  4. ‘attempts‘
  5. ‘nextRunAt‘
  6. ‘status‘
  7. ‘lastError‘
  8. ‘createdAt‘
  9. ‘updatedAt‘
  10. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘jobType‘: String, defaults to ‘’unknown’‘.
  - ‘payload‘: String, JSON-serialized object, defaults to ‘’{}’‘ from ‘JSON.stringify(payload.pay
load || {})‘.
  - ‘attempts‘: Integer-like string. Stored as ‘String(payload.attempts || 0)‘ and incremented as
strings.
  - ‘nextRunAt‘: DateTime, defaults to current timestamp if omitted.
  - ‘status‘: String, observed enum values: ‘queued‘, ‘dead_letter‘.
  - ‘lastError‘: String, optional, defaults to empty string.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** none.
- **Soft delete:** enabled via ‘deletedAt‘.
- **Observed active-record filter:** implicit repository ‘deletedAt‘ filter; retry scheduler addit
ionally filters ‘status === ’queued’‘.
- **AppSheet consumer evidence:** none found.

Table: ‘audit_log‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘audit_log‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘actor‘
  3. ‘action‘
  4. ‘resourceType‘
  5. ‘resourceId‘
  6. ‘status‘
  7. ‘message‘
  8. ‘metadata‘
  9. ‘createdAt‘
  10. ‘updatedAt‘
  11. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘actor‘: String, observed values: ‘system‘, ‘scheduler‘.

  - ‘action‘: String, required by application writes.
  - ‘resourceType‘: String, observed values: ‘slack_host‘, ‘job‘.
  - ‘resourceId‘: String, optional, commonly empty string.
  - ‘status‘: String, observed enum values: ‘info‘, ‘error‘ plus caller-supplied job status.
  - ‘message‘: String, optional human-readable audit message.
  - ‘metadata‘: String, JSON-serialized object.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, present in current implementation even though this is an audit/log tabl
e.
- **Foreign keys:** none.
- **Soft delete:** implementation supports ‘deletedAt‘, but this table is logically append-only /
audit-oriented.
- **Audit/log classification:** yes â should be treated as immutable in downstream design even tho
ugh current generic repository includes ‘deletedAt‘.
- **AppSheet consumer evidence:** none found.

Table: ‘students‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘students‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘name‘
  3. ‘source‘
  4. ‘createdAt‘
  5. ‘updatedAt‘
  6. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key, explicitly required by schema and caller-supplied in host flows.
  - ‘name‘: String, required by schema.
  - ‘source‘: String, optional with schema default ‘’slack’’; observed values include ‘slash_comma
nd‘, ‘workflow_webhook‘.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** none.
- **Soft delete:** enabled via ‘deletedAt‘.
- **AppSheet consumer evidence:** none found.

Table: ‘enrollments‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘enrollments‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘studentId‘
  3. ‘courseId‘
  4. ‘status‘
  5. ‘source‘
  6. ‘createdAt‘
  7. ‘updatedAt‘
  8. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘studentId‘: String, required by schema, foreign key to ‘students.id‘.
  - ‘courseId‘: String, required by schema.
  - ‘status‘: String, optional with schema default ‘’enrolled’’; observed enum values: ‘enrolled‘
and workflow-provided overrides.
  - ‘source‘: String, optional with schema default ‘’slack’’; observed values: ‘slash_command‘, ‘w
orkflow_webhook‘.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.

- **Foreign keys:** ‘studentId -> students.id‘.
- **Soft delete:** enabled via ‘deletedAt‘.
- **AppSheet consumer evidence:** none found.

Table: ‘automations‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘automations‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘requestId‘
  3. ‘workflow‘
  4. ‘eventType‘
  5. ‘state‘
  6. ‘createdAt‘
  7. ‘updatedAt‘
  8. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘requestId‘: String, required by schema.
  - ‘workflow‘: String, required by schema, defaults in service to ‘’unknown_workflow‘‘ before ins
ert path.
  - ‘eventType‘: String, required by schema, defaults in service to ‘’workflow_event‘‘ before inse
rt path.
  - ‘state‘: String, optional with schema default ‘’received‘‘; observed enum values: ‘received‘,
‘queued‘ and payload-driven values.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, optional soft-delete marker.
- **Foreign keys:** none.
- **Soft delete:** enabled via ‘deletedAt‘.
- **AppSheet consumer evidence:** none found.

Table: ‘audit_logs‘
- **Origin:** ‘db.schema()‘
- **Sheet/tab name:** ‘audit_logs‘
- **Column order confidence:** exact from schema registration
- **Columns (ordered):**
  1. ‘id‘
  2. ‘requestId‘
  3. ‘source‘
  4. ‘action‘
  5. ‘status‘
  6. ‘message‘
  7. ‘metadata‘
  8. ‘createdAt‘
  9. ‘updatedAt‘
  10. ‘deletedAt‘
- **Column details:**
  - ‘id‘: String, primary key. Auto-generated when omitted.
  - ‘requestId‘: String, required by schema.
  - ‘source‘: String, required by schema; observed values include route types such as ‘slash_comma
nd‘ / ‘workflow_webhook‘.
  - ‘action‘: String, required by schema; observed values include ‘enrollment_upsert‘, ‘automation
_upsert‘, route type names.
  - ‘status‘: String, required by schema; observed enum values: ‘success‘, ‘failure‘.
  - ‘message‘: String, optional.
  - ‘metadata‘: String, optional JSON string.
  - ‘createdAt‘: DateTime, auto-populated.
  - ‘updatedAt‘: DateTime, auto-populated.
  - ‘deletedAt‘: DateTime, present in implementation even though this is an audit/log table.
- **Foreign keys:** none.
- **Soft delete:** implementation supports ‘deletedAt‘, but the table is logically append-only / a
udit-oriented.

- **Audit/log classification:** yes.
- **AppSheet consumer evidence:** none found.

Section 3 â Cross-cutting Patterns

Header / column-order reconstruction notes
- All discovered tables had explicit ‘db.schema(...)‘ registrations, so column order is known exac
tly from code.
- No table required fallback reconstruction from ‘data[0]‘ or ‘headers.indexOf(...)‘.

Generic SheetDb persistence behavior
- ‘id‘ is the effective primary key for every registered table.
- ‘createdAt‘, ‘updatedAt‘, and ‘deletedAt‘ are conventional system columns configured through ‘Co
nfig‘ / script properties.
- Insert behavior:
  - auto-generates ‘id‘ when missing
  - auto-populates ‘createdAt‘
  - always refreshes ‘updatedAt‘
  - initializes ‘deletedAt‘ to ‘’’‘ when omitted
- ‘findAll()‘ only returns rows where ‘deletedAt‘ is blank, confirming soft-delete behavior is act
ive globally.

Soft-delete and audit notes
- The current codebase uses ‘deletedAt‘, not ‘_rowStatus‘.
- No table in this repo currently uses ‘_rowStatus = ’active’‘ filtering.
- Audit/log tables (‘audit_log‘, ‘audit_logs‘) still include ‘deletedAt‘ because the generic schem
a scaffold uses the same trailing system columns for all tables. This differs from the requested f
uture rule of avoiding delete fields on audit/log tables.

AppSheet consumer scan
- No direct AppSheet integration, AppConfig, webhook handler, ‘_Key‘ formula column, read-only App
Sheet annotations, or formula-column comments were found.
- Therefore no table can be positively flagged as AppSheet-consumed from current code.

Section 4 â Ready-to-paste SheetDb schema block

’’’javascript
function registerRwrLmsSchemas(db) {
  db.schema(’learners’, {
    columns: [’id’, ’slackUserId’, ’email’, ’name’, ’status’, ’createdAt’, ’updatedAt’, ’deletedAt
’],
    fields: {
      id: { type: ’string’ },
      slackUserId: { type: ’string’ },
      email: { type: ’string’ },
      name: { type: ’string’ },
      status: { type: ’string’ }
    }
  });

  db.schema(’enrollment’, {
    columns: [’id’, ’learnerId’, ’courseId’, ’track’, ’status’, ’createdAt’, ’updatedAt’, ’deleted
At’],
    fields: {
      id: { type: ’string’ },
      learnerId: { type: ’string’ },
      courseId: { type: ’string’ },
      track: { type: ’string’, default: ’ONBOARDING’ },
      status: { type: ’string’ }
    }
  });

  db.schema(’lessons’, {
    columns: [’id’, ’courseId’, ’track’, ’title’, ’contentRef’, ’active’, ’createdAt’, ’updatedAt’

```
, 'deletedAt'],
    fields: {
      id: { type: 'string' },
      courseId: { type: 'string' },
      track: { type: 'string' },
      title: { type: 'string' },
      contentRef: { type: 'string' },
      active: { type: 'string', default: 'true' }
    }
  });

  db.schema('learner_progress', {
    columns: ['id', 'learnerId', 'lessonId', 'state', 'dueAt', 'completedAt', 'createdAt', 'update
dAt', 'deletedAt'],
    fields: {
      id: { type: 'string' },
      learnerId: { type: 'string' },
      lessonId: { type: 'string' },
      state: { type: 'string' },
      dueAt: { type: 'string' },
      completedAt: { type: 'string' }
    }
  });

  db.schema('submission_log', {
    columns: ['id', 'learnerId', 'lessonId', 'submitKey', 'payload', 'createdAt', 'updatedAt', 'de
letedAt'],
    fields: {
      id: { type: 'string' },
      learnerId: { type: 'string' },
      lessonId: { type: 'string' },
      submitKey: { type: 'string' },
      payload: { type: 'string', default: '' }
    }
  });

  db.schema('delivery_queue', {
    columns: ['id', 'learnerId', 'lessonId', 'status', 'runAt', 'createdAt', 'updatedAt', 'deleted
At'],
    fields: {
      id: { type: 'string' },
      learnerId: { type: 'string' },
      lessonId: { type: 'string' },
      status: { type: 'string' },
      runAt: { type: 'string' }
    }
  });

  db.schema('retry_queue', {
    columns: ['id', 'jobType', 'payload', 'attempts', 'nextRunAt', 'status', 'lastError', 'created
At', 'updatedAt', 'deletedAt'],
    fields: {
      id: { type: 'string' },
      jobType: { type: 'string', default: 'unknown' },
      payload: { type: 'string', default: '{}' },
      attempts: { type: 'string', default: '0' },
      nextRunAt: { type: 'string' },
      status: { type: 'string', default: 'queued' },
      lastError: { type: 'string', default: '' }
    }
  });

  db.schema('audit_log', {
    columns: ['id', 'actor', 'action', 'resourceType', 'resourceId', 'status', 'message', 'metadat
```

```
a', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      id: { type: 'string' },
      actor: { type: 'string' },
      action: { type: 'string' },
      resourceType: { type: 'string' },
      resourceId: { type: 'string', default: '' },
      status: { type: 'string' },
      message: { type: 'string' },
      metadata: { type: 'string', default: '{}' }
    }
  });

  db.schema('students', {
    columns: ['id', 'name', 'source', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      id: { required: true, type: 'string' },
      name: { required: true, type: 'string' },
      source: { type: 'string', default: 'slack' }
    }
  });

  db.schema('enrollments', {
    columns: ['id', 'studentId', 'courseId', 'status', 'source', 'createdAt', 'updatedAt', 'delete
dAt'],
    fields: {
      id: { type: 'string' },
      studentId: { required: true, type: 'string' },
      courseId: { required: true, type: 'string' },
      status: { type: 'string', default: 'enrolled' },
      source: { type: 'string', default: 'slack' }
    }
  });

  db.schema('automations', {
    columns: ['id', 'requestId', 'workflow', 'eventType', 'state', 'createdAt', 'updatedAt', 'dele
tedAt'],
    fields: {
      id: { type: 'string' },
      requestId: { required: true, type: 'string' },
      workflow: { required: true, type: 'string' },
      eventType: { required: true, type: 'string' },
      state: { type: 'string', default: 'received' }
    }
  });

  db.schema('audit_logs', {
    columns: ['id', 'requestId', 'source', 'action', 'status', 'message', 'metadata', 'createdAt',
 'updatedAt', 'deletedAt'],
    fields: {
      id: { type: 'string' },
      requestId: { required: true, type: 'string' },
      source: { required: true, type: 'string' },
      action: { required: true, type: 'string' },
      status: { required: true, type: 'string' },
      message: { type: 'string' },
      metadata: { type: 'string' }
    }
  });

  return db;
}
'''
```

Section 5 â Search evidence map

Search patterns used
- ‘*.csv‘
- ‘db.schema(‘
- ‘registerTable(‘
- ‘SpreadsheetApp.getSheetByName(‘
- ‘getDataRange().getValues()‘
- ‘headers.indexOf(‘
- ‘db.table(‘
- ‘AppSheet‘, ‘_Key‘, ‘formula‘, ‘_rowStatus‘

Findings summary by requested search class
- **Physical CSV files:** none found.
- **‘SheetDb.registerTable()‘ calls:** library bootstrap registers arbitrary tables, but consuming app code uses ‘db.schema(...)‘ rather than direct ‘registerTable(...)‘.
- **‘db.schema()‘ calls:** found in ‘00_WebAppEntry.gs‘, ‘slack_host/00_WebAppEntry.gs‘, and ‘slack_host/11_HostTestHarness.gs‘.
- **‘SpreadsheetApp.getSheetByName()‘ calls:** only in ‘24_SheetsGateway.gs‘ and mirrored library copy.
- **‘sheet.getDataRange().getValues()‘ calls:** only in ‘24_SheetsGateway.gs‘ and mirrored library copy.
- **‘headers.indexOf()‘ calls:** none found.
- **‘db.table(’...’)‘ calls:** found across LMS services, health monitoring, retry resolution, Slack host services, and router logic.

FILE PATH: docs/generated_csv/audit_log.csv

```
id,actor,action,resourceType,resourceId,status,message,metadata,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/audit_logs.csv

```
id,requestId,source,action,status,message,metadata,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/automations.csv

```
id,requestId,workflow,eventType,state,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/delivery_queue.csv

```
id,learnerId,lessonId,status,runAt,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/enrollment.csv

```
id,learnerId,courseId,track,status,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/enrollments.csv

```
id,studentId,courseId,status,source,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/learner_progress.csv

```
id,learnerId,lessonId,state,dueAt,completedAt,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/learners.csv

```
id,slackUserId,email,name,status,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/lessons.csv

```
id,courseId,track,title,contentRef,active,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/retry_queue.csv

```
id,jobType,payload,attempts,nextRunAt,status,lastError,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/students.csv

```
id,name,source,createdAt,updatedAt,deletedAt
```

FILE PATH: docs/generated_csv/submission_log.csv

```
id,learnerId,lessonId,submitKey,payload,createdAt,updatedAt,deletedAt
```

FILE PATH: 00_WebAppEntry.gs

```
/**
 * Host dependency wiring and DB client bootstrap.
 */

function createHostDbClient(config) {
  var cfg = config || ConfigBootstrap.load();
  var db = SheetDb.createClient({ spreadsheetId: cfg.spreadsheetId });
  var schema = function(columns) { return { columns: columns, fields: {} }; };

  db.schema('learners', schema(['id', 'slackUserId', 'email', 'name', 'status', 'createdAt', 'upda
tedAt', 'deletedAt']));
  db.schema('enrollment', schema(['id', 'learnerId', 'courseId', 'track', 'status', 'createdAt', '
updatedAt', 'deletedAt']));
  db.schema('lessons', schema(['id', 'courseId', 'track', 'title', 'contentRef', 'active', 'create
dAt', 'updatedAt', 'deletedAt']));
  db.schema('learner_progress', schema(['id', 'learnerId', 'lessonId', 'state', 'dueAt', 'complete
dAt', 'createdAt', 'updatedAt', 'deletedAt']));
  db.schema('submission_log', schema(['id', 'learnerId', 'lessonId', 'submitKey', 'payload', 'crea
tedAt', 'updatedAt', 'deletedAt']));
  db.schema('delivery_queue', schema(['id', 'learnerId', 'lessonId', 'status', 'runAt', 'createdAt
', 'updatedAt', 'deletedAt']));
  db.schema('retry_queue', schema(['id', 'jobType', 'payload', 'attempts', 'nextRunAt', 'status',
'lastError', 'createdAt', 'updatedAt', 'deletedAt']));
  db.schema('audit_log', schema(['id', 'actor', 'action', 'resourceType', 'resourceId', 'status',
'message', 'metadata', 'createdAt', 'updatedAt', 'deletedAt']));

  return db;
}

function createHostRouter() {
  return SlackRouter;
}

function createHostDependencies(config) {
  var cfg = config || ConfigBootstrap.load();
  var db = createHostDbClient(cfg);
  var blocks = new SlackBlockKitBuilder();
  var stateMachine = new LearnerProgressStateMachine();
  var retryResolver = new RetryResolver(db, cfg);
  var slackApi = new SlackApiClient(cfg, retryResolver);

  var deps = {
    config: cfg,
    db: db,
    parser: SlackPayloadParser,
    security: SlackSecurity,
    router: createHostRouter(),
    slackApiClient: slackApi,
    blockKitBuilder: blocks,
    stateMachine: stateMachine,
    enrollmentService: new LmsEnrollmentService(db, slackApi, blocks, cfg),
    lessonService: new LmsLessonService(db, slackApi, blocks, stateMachine, cfg),
    progressService: new LmsProgressService(db, slackApi, blocks, cfg),
    completionService: new LmsCompletionService(db, slackApi, blocks, stateMachine, cfg),
    reminderService: new LmsReminderService(db, slackApi, blocks, cfg),
    reportService: new LmsReportService(db, cfg),
    healthMonitor: new HealthMonitor(db, cfg),
    retryResolver: retryResolver,
    audit: function(action, metadata) {
      db.table('audit_log').insert({
        actor: 'system',
        action: action,
```

```
        resourceType: 'slack_host',
        resourceId: '',
        status: 'info',
        message: action,
        metadata: JSON.stringify(metadata || {})
      });
      db.audit(action, 'audit_log', metadata || {});
    }
  };

  deps.slackService = new SlackService(
    deps.lessonService,
    deps.completionService,
    deps.progressService,
    deps.enrollmentService,
    deps.reportService,
    deps.blockKitBuilder
  );

  return deps;
}
```

FILE PATH: 01_SlackRouter.gs

```
/**
 * Central request router for normalized Slack envelopes.
 */
var SlackRouter = {
  route: function(parsed, deps) {
    if (!parsed || !parsed.ok) {
      return { ok: false, code: 'PARSE_FAILED', response: { ok: false, error: 'invalid_request' }
};
    }

    if (parsed.routeType === 'url_verification') {
      return { ok: true, code: 'URL_VERIFICATION', response: { challenge: parsed.body.challenge }
};
    }

    if (parsed.routeType === 'slash_command') {
      return { ok: true, code: 'SLASH_OK', response: deps.slackService.handleSlashCommand(parsed)
};
    }

    if (parsed.routeType === 'interactivity') {
      return { ok: true, code: 'INTERACTIVITY_OK', response: deps.slackService.handleInteractivity
(parsed) };
    }

    if (parsed.routeType === 'event_callback') {
      return { ok: true, code: 'EVENT_OK', response: deps.slackService.handleEventCallback(parsed)
 };
    }

    if (parsed.routeType === 'workflow_webhook') {
      return { ok: true, code: 'WORKFLOW_OK', response: deps.slackService.handleWorkflowWebhook(pa
rsed) };
    }

    return {
      ok: false,
      code: 'UNSUPPORTED_ROUTE',
      response: { ok: false, error: 'unsupported_route', routeType: parsed.routeType }
    };
  }
};
```

FILE PATH: 02_SlackPayloadParser.gs

```
/**
 * Normalizes Slack requests to a single envelope.
 */
var SlackPayloadParser = {
  parse: function(e) {
    var rawBody = (e && e.postData && e.postData.contents) ? String(e.postData.contents) : '';
    var params = (e && e.parameter) ? e.parameter : {};
    var headers = (e && e.headers) ? e.headers : {};
    var body = this._parseJson(rawBody);
    var interaction = params.payload ? this._parseJson(params.payload) : null;
    var parseOk = true;

    if (!body && rawBody && String((e.postData && e.postData.type) || '').indexOf('application/jso
n') !== -1) {
      parseOk = false;
    }

    body = body || {};

    var routeType = 'unknown';
    if (body.type === 'url_verification') routeType = 'url_verification';
    else if (params.command) routeType = 'slash_command';
    else if (interaction) routeType = 'interactivity';
    else if (body.type === 'event_callback') routeType = 'event_callback';
    else if (body.workflow_step || params.workflow) routeType = 'workflow_webhook';

    var source = interaction || body || params;
    return {
      ok: parseOk,
      routeType: routeType,
      rawBody: rawBody,
      body: body,
      params: params,
      command: String(params.command || ''),
      payloadType: String((interaction && interaction.type) || body.type || ''),
      userId: this._pick(source, ['user.id', 'user_id', 'event.user']),
      channelId: this._pick(source, ['channel.id', 'channel_id', 'event.channel']),
      teamId: this._pick(source, ['team.id', 'team_id']),
      triggerId: this._pick(source, ['trigger_id']),
      responseUrl: this._pick(source, ['response_url']),
      slackSignature: String(headers['x-slack-signature'] || headers['X-Slack-Signature'] || param
s.x_slack_signature || ''),
      slackTimestamp: String(headers['x-slack-request-timestamp'] || headers['X-Slack-Request-Time
stamp'] || params.x_slack_request_timestamp || ''),
      interaction: interaction,
      parseError: parseOk ? '' : 'Invalid JSON body'
    };
  },

  _parseJson: function(text) {
    try { return JSON.parse(text); } catch (err) { return null; }
  },

  _pick: function(obj, paths) {
    var i;
    for (i = 0; i < paths.length; i++) {
      var value = this._get(obj, paths[i]);
      if (value) return String(value);
    }
    return '';
  },
```

```
  _get: function(obj, path) {
    return String(path || ’’).split(’.’).reduce(function(acc, key) {
      return (acc && acc[key] != null) ? acc[key] : ’’;
    }, obj || {});
  }
};
```

FILE PATH: 03_SlackService.gs

```
/**
 * Slack dispatch layer for slash/event/interactivity registries.
 */
class SlackService {
  constructor(lessonService, completionService, progressService, enrollmentService, reportService,
 blocks) {
    this._lesson = lessonService;
    this._completion = completionService;
    this._progress = progressService;
    this._enrollment = enrollmentService;
    this._report = reportService;
    this._blocks = blocks;
    this._slashRegistry = {
      '/lesson': this._lesson.handleLesson.bind(this._lesson),
      '/submit': this._completion.handleSubmit.bind(this._completion),
      '/progress': this._progress.handleProgress.bind(this._progress),
      '/help': this._handleHelp.bind(this),
      '/enroll': this._handleEnroll.bind(this),
      '/report': this._handleReport.bind(this)
    };
  }

  handleSlashCommand(parsed) {
    if (!parsed || !parsed.command) {
      return { response_type: 'ephemeral', text: 'Missing slash command context.' };
    }
    var handler = this._slashRegistry[parsed.command];
    if (!handler) return { response_type: 'ephemeral', text: 'Unsupported command: ' + parsed.comm
and };
    return handler(parsed);
  }

  handleInteractivity(parsed) {
    var payload = parsed.interaction || {};
    var actionId = (payload.actions && payload.actions[0] && payload.actions[0].action_id) || '';
    var callbackId = payload.callback_id || '';
    var type = payload.type || '';

    if (type === 'view_submission') {
      return { response_action: 'clear' };
    }
    if (actionId || callbackId) {
      return { response_type: 'ephemeral', text: 'Interactive action received.' };
    }
    return { response_type: 'ephemeral', text: 'Unsupported interactive payload.' };
  }

  handleEventCallback(parsed) {
    var event = (parsed.body && parsed.body.event) ? parsed.body.event : {};
    if (event.type === 'app_mention') {
      return { ok: true, text: 'Thanks for the mention. Try /lesson, /submit, or /progress.' };
    }
    if (event.type === 'message' && event.channel_type === 'im') {
      return { ok: true, text: 'DM received. Use /lesson to get started.' };
    }
    return { ok: true, text: 'Event ignored', eventType: event.type || '' };
  }

  handleWorkflowWebhook(parsed) {
    return { ok: true, text: 'Workflow webhook received', payloadType: parsed.payloadType };
  }
```

```
  _handleHelp() {
    return { response_type: 'ephemeral', text: 'Commands: /lesson, /submit <lessonId>, /progress' };
  }

  _handleEnroll(parsed) {
    return this._enrollment.enrollLearner({
      slackUserId: parsed.userId,
      courseId: parsed.params.text || ''
    });
  }

  _handleReport() {
    var summary = this._report.buildWeeklySummary();
    return { response_type: 'ephemeral', text: 'Weekly summary', blocks: this._blocks.buildAdminSummary(summary) };
  }
}
```

FILE PATH: 04_SlackSecurity.gs

```
/**
 * Slack security verifier (signing secret + replay + fallback mode).
 */
var SlackSecurity = {
  verify: function(parsed, config) {
    if (!parsed || !parsed.ok) {
      return { ok: false, code: 'PARSE_FAILED', message: parsed && parsed.parseError ? parsed.pars
eError : 'Invalid payload' };
    }

    var secret = String((config && config.slackSigningSecret) || '');
    var signature = String(parsed.slackSignature || '');
    var ts = String(parsed.slackTimestamp || '');

    if (secret && signature && ts && parsed.rawBody) {
      return this._verifySignature(secret, signature, ts, parsed.rawBody);
    }

    var token = String((config && config.slackVerificationToken) || '');
    if (token) {
      var presented = String((parsed.params && parsed.params.token) || (parsed.body && parsed.body
.token) || '');
      if (this._constantTimeEquals(token, presented)) {
        return { ok: true, code: 'OK_FALLBACK_TOKEN', message: 'Fallback token verification passed
.' };
      }
      return { ok: false, code: 'FALLBACK_TOKEN_MISMATCH', message: 'Fallback token verification f
ailed.' };
    }

    return { ok: false, code: 'UNVERIFIED', message: 'Slack signature headers unavailable and fall
back not configured.' };
  },

  _verifySignature: function(secret, signature, ts, rawBody) {
    var tsNum = Number(ts);
    if (!tsNum || Math.abs(Math.floor(Date.now() / 1000) - tsNum) > 300) {
      return { ok: false, code: 'REPLAY_REJECTED', message: 'Timestamp outside replay window.' };
    }

    var base = 'v0:' + ts + ':' + rawBody;
    var bytes = Utilities.computeHmacSha256Signature(base, secret);
    var hex = bytes.map(function(b) {
      var n = (b < 0 ? b + 256 : b).toString(16);
      return n.length === 1 ? '0' + n : n;
    }).join('');
    var expected = 'v0=' + hex;

    if (!this._constantTimeEquals(expected, signature)) {
      return { ok: false, code: 'SIGNATURE_MISMATCH', message: 'Slack signature mismatch.' };
    }

    return { ok: true, code: 'OK', message: 'Signature verified.' };
  },

  _constantTimeEquals: function(a, b) {
    var aa = String(a || '');
    var bb = String(b || '');
    if (!aa || !bb || aa.length !== bb.length) return false;
    var mismatch = 0;
    for (var i = 0; i < aa.length; i++) mismatch |= (aa.charCodeAt(i) ^ bb.charCodeAt(i));
    return mismatch === 0;
```

```
  }
};
```

FILE PATH: 05_LmsEnrollmentService.gs

```
class LmsEnrollmentService {
  constructor(db, slackApiClient, blocks, config) {
    this._db = db;
    this._slack = slackApiClient;
    this._blocks = blocks;
    this._config = config || {};
  }

  enrollLearner(input) {
    var learner = this.ensureLearnerRecord(input || {});
    var enrollment = this.ensureEnrollmentRecord({
      learnerId: learner.id,
      courseId: (input && input.courseId) || this._config.defaultCourseId,
      track: this._config.defaultTrack
    });
    var queued = this.queueFirstLesson({ learnerId: learner.id, courseId: enrollment.courseId });
    this.sendWelcomeDm({ slackUserId: learner.slackUserId, learnerId: learner.id, courseId: enroll
ment.courseId });
    this._db.audit('enroll_learner', 'enrollment', { learnerId: learner.id, enrollmentId: enrollme
nt.id });
    return { ok: true, code: 'ENROLLED', learnerId: learner.id, enrollmentId: enrollment.id, queue
Id: queued.id };
  }

  findLearnerBySlackUserId(slackUserId) {
    if (!slackUserId) return null;
    return this._db.table('learners').findAll().filter(function(row) { return row.slackUserId ===
slackUserId; })[0] || null;
  }

  ensureLearnerRecord(input) {
    var existing = this.findLearnerBySlackUserId(input.slackUserId);
    if (existing) return existing;
    return this._db.table('learners').insert({
      slackUserId: input.slackUserId,
      email: input.email || '',
      name: input.name || '',
      status: 'active'
    });
  }

  ensureEnrollmentRecord(input) {
    var existing = this._db.table('enrollment').findAll().filter(function(row) {
      return row.learnerId === input.learnerId && row.courseId === input.courseId;
    })[0];
    if (existing) return existing;
    return this._db.table('enrollment').insert({
      learnerId: input.learnerId,
      courseId: input.courseId,
      track: input.track || 'ONBOARDING',
      status: 'active'
    });
  }

  sendWelcomeDm(input) {
    var dm = this._slack.openDm(input.slackUserId);
    if (!dm.ok) return dm;
    return this._slack.postMessage(dm.channelId, 'Welcome to RWR LMS', this._blocks.buildWelcomeMe
ssage(input));
  }

  queueFirstLesson(input) {
```

```
    // TODO: resolve first lesson id from runtime lesson sequencing rules.
    return this._db.table('delivery_queue').insert({
      learnerId: input.learnerId,
      lessonId: '',
      status: 'queued',
      runAt: new Date().toISOString()
    });
  }
}
```

FILE PATH: 06_LmsLessonService.gs

```
class LmsLessonService {
  constructor(db, slackApiClient, blocks, stateMachine, config) {
    this._db = db;
    this._slack = slackApiClient;
    this._blocks = blocks;
    this._state = stateMachine;
    this._config = config || {};
  }

  handleLesson(ctx) {
    var current = this.getCurrentLessonForLearner(ctx.userId);
    if (!current.ok) return { response_type: 'ephemeral', text: current.message };
    return {
      response_type: 'ephemeral',
      text: 'Your current lesson',
      blocks: this._blocks.buildLessonCard(current.lesson)
    };
  }

  getCurrentLessonForLearner(slackUserId) {
    var learner = this._db.table('learners').findAll().filter(function(r) { return r.slackUserId =
== slackUserId; })[0];
    if (!learner) return { ok: false, code: 'LEARNER_NOT_FOUND', message: 'Learner not found.' };

    var progress = this._db.table('learner_progress').findAll().filter(function(r) {
      return r.learnerId === learner.id && r.state !== 'completed';
    })[0];
    if (!progress) return { ok: false, code: 'NO_ACTIVE_LESSON', message: 'No lesson assigned yet.
' };

    var lesson = this._db.table('lessons').findById(progress.lessonId) || { id: progress.lessonId,
 title: 'Lesson', track: '' };
    return { ok: true, learnerId: learner.id, lesson: lesson, progress: progress };
  }

  deliverLessonToLearner(learnerId, lessonId) {
    var learner = this._db.table('learners').findById(learnerId);
    if (!learner) return { ok: false, code: 'LEARNER_NOT_FOUND', message: 'Learner not found' };

    var dm = this._slack.openDm(learner.slackUserId);
    if (!dm.ok) return dm;
    var sent = this._slack.postMessage(dm.channelId, 'New lesson available', this._blocks.buildLes
sonCard({ lessonId: lessonId }));
    if (!sent.ok) return sent;

    this._db.table('delivery_queue').insert({ learnerId: learnerId, lessonId: lessonId, status: 'd
elivered', runAt: new Date().toISOString() });
    var active = this._db.table('learner_progress').findAll().filter(function(r) { return r.learne
rId === learnerId && r.lessonId === lessonId; })[0];
    if (active) this._db.table('learner_progress').update(active.id, { state: 'delivered' });
    return { ok: true, code: 'DELIVERED', learnerId: learnerId, lessonId: lessonId };
  }

  deliverPendingLessons() {
    var pending = this._db.table('delivery_queue').findAll().filter(function(row) { return row.sta
tus === 'queued'; });
    var delivered = [];
    for (var i = 0; i < pending.length; i++) {
      var res = this.deliverLessonToLearner(pending[i].learnerId, pending[i].lessonId);
      delivered.push({ id: pending[i].id, ok: !!res.ok });
      if (res.ok) this._db.table('delivery_queue').update(pending[i].id, { status: 'delivered' });
    }
```

```
    return { ok: true, total: pending.length, delivered: delivered };
  }
}
```

FILE PATH: 07_LmsProgressService.gs

```
class LmsProgressService {
  constructor(db, slackApiClient, blocks, config) {
    this._db = db;
    this._slack = slackApiClient;
    this._blocks = blocks;
    this._config = config || {};
  }

  handleProgress(ctx) {
    var snapshot = this.getProgressSnapshot(ctx.userId);
    if (!snapshot.ok) return { response_type: 'ephemeral', text: snapshot.message };
    return {
      response_type: 'ephemeral',
      text: 'Progress summary',
      blocks: this._blocks.buildProgressSummary(snapshot)
    };
  }

  getProgressSnapshot(slackUserId) {
    var learner = this._db.table('learners').findAll().filter(function(row) { return row.slackUser
Id === slackUserId; })[0];
    if (!learner) return { ok: false, code: 'LEARNER_NOT_FOUND', message: 'Learner not found.' };

    var progress = this._db.table('learner_progress').findAll().filter(function(row) { return row.
learnerId === learner.id; });
    var completed = progress.filter(function(r) { return r.state === 'completed'; }).length;
    var overdue = progress.filter(function(r) { return r.state === 'overdue'; }).length;
    var active = progress.filter(function(r) { return r.state !== 'completed'; })[0] || null;

    return {
      ok: true,
      learnerId: learner.id,
      activeCourse: this._config.defaultCourseId || '',
      currentLesson: active ? active.lessonId : '',
      completedCount: completed,
      overdueCount: overdue,
      nextAction: active ? 'Complete lesson ' + active.lessonId : 'Request next lesson',
      totalCount: progress.length
    };
  }
}
```

FILE PATH: 08_LmsCompletionService.gs

```
class LmsCompletionService {
  constructor(db, slackApiClient, blocks, stateMachine, config) {
    this._db = db;
    this._slack = slackApiClient;
    this._blocks = blocks;
    this._state = stateMachine;
    this._config = config || {};
  }

  handleSubmit(ctx) {
    var lessonId = String((ctx.params && ctx.params.text) || '').trim();
    if (!lessonId) return { response_type: 'ephemeral', text: 'Usage: /submit <lesson_id>' };

    var recorded = this.recordSubmission({ slackUserId: ctx.userId, lessonId: lessonId, payload: c
tx.rawBody });
    if (!recorded.ok) return { response_type: 'ephemeral', text: recorded.message };

    this.advanceLessonState({ learnerProgressId: recorded.learnerProgressId, toState: 'submitted'
});
    this.queueNextLesson({ learnerId: recorded.learnerId, currentLessonId: lessonId });
    return { response_type: 'ephemeral', text: 'Submission received for ' + lessonId };
  }

  recordSubmission(input) {
    var learner = this._db.table('learners').findAll().filter(function(r) { return r.slackUserId =
== input.slackUserId; })[0];
    if (!learner) return { ok: false, code: 'LEARNER_NOT_FOUND', message: 'Learner not found.' };

    var submitKey = learner.id + ':' + input.lessonId;
    var existing = this._db.table('submission_log').findAll().filter(function(r) { return r.submit
Key === submitKey; })[0];
    if (existing) {
      return { ok: true, code: 'DUPLICATE_SUBMISSION', learnerId: learner.id, learnerProgressId: l
earner.id, submissionId: existing.id, message: 'Submission already recorded.' };
    }

    var row = this._db.table('submission_log').insert({
      learnerId: learner.id,
      lessonId: input.lessonId,
      submitKey: submitKey,
      payload: input.payload || ''
    });

    this._db.audit('record_submission', 'submission_log', { learnerId: learner.id, lessonId: input
.lessonId, submissionId: row.id });
    return { ok: true, code: 'SUBMISSION_RECORDED', learnerId: learner.id, learnerProgressId: lear
ner.id, submissionId: row.id };
  }

  advanceLessonState(input) {
    var progress = this._db.table('learner_progress').findById(input.learnerProgressId);
    if (!progress) return { ok: false, code: 'PROGRESS_NOT_FOUND', message: 'Progress not found.'
};
    var transition = this._state.transition(progress, input.toState, { source: 'submission' });
    if (!transition.ok) return transition;
    this._db.table('learner_progress').update(progress.id, { state: transition.record.state });
    return { ok: true, code: 'STATE_UPDATED', recordId: progress.id };
  }

  queueNextLesson(input) {
    // TODO: determine next lesson ordering from course track.
    var row = this._db.table('delivery_queue').insert({
```

```
      learnerId: input.learnerId,
      lessonId: '',
      status: 'queued',
      runAt: new Date().toISOString()
    });
    return { ok: true, code: 'NEXT_QUEUED', queueId: row.id };
  }
}
```

FILE PATH: 09_LmsReminderService.gs

```
class LmsReminderService {
  constructor(db, slackApiClient, blocks, config) {
    this._db = db;
    this._slack = slackApiClient;
    this._blocks = blocks;
    this._config = config || {};
  }

  sendOverdueReminders() {
    var overdue = this._db.table('learner_progress').findAll().filter(function(row) { return row.s
tate === 'overdue'; });
    var results = [];
    for (var i = 0; i < overdue.length; i++) {
      results.push(this.sendReminderForLearner(overdue[i].learnerId));
    }
    return { ok: true, total: overdue.length, results: results };
  }

  sendReminderForLearner(learnerId) {
    var learner = this._db.table('learners').findById(learnerId);
    if (!learner) return { ok: false, code: 'LEARNER_NOT_FOUND', learnerId: learnerId };
    var dm = this._slack.openDm(learner.slackUserId);
    if (!dm.ok) return dm;
    return this._slack.postMessage(dm.channelId, 'Lesson reminder', this._blocks.buildReminder({ l
earnerId: learnerId }));
  }

  escalateStalledLearner(learnerId) {
    return {
      ok: true,
      learnerId: learnerId,
      escalated: !!this._config.opsAlertChannel,
      message: this._config.opsAlertChannel ? 'Escalation target configured.' : 'No ops channel co
nfigured.'
    };
  }
}
```

FILE PATH: 10_LmsReportService.gs

```
class LmsReportService {
  constructor(db, config) {
    this._db = db;
    this._config = config || {};
  }

  buildWeeklySummary() {
    var learners = this._db.table('learners').findAll().length;
    var progress = this._db.table('learner_progress').findAll();
    return {
      ok: true,
      learners: learners,
      completed: progress.filter(function(r) { return r.state === 'completed'; }).length,
      submitted: progress.filter(function(r) { return r.state === 'submitted'; }).length
    };
  }

  buildCohortSummary(courseId) {
    var rows = this._db.table('enrollment').findAll().filter(function(r) { return r.courseId === c
ourseId; });
    return { ok: true, courseId: courseId, enrollmentCount: rows.length, activeCount: rows.filter(
function(r) { return r.status === 'active'; }).length };
  }

  buildOverdueSummary() {
    var rows = this._db.table('learner_progress').findAll().filter(function(r) { return r.state ==
= 'overdue'; });
    return { ok: true, overdueCount: rows.length, learnerIds: rows.map(function(r) { return r.lear
nerId; }) };
  }
}
```

FILE PATH: 11_SlackApiClient.gs

```
class SlackApiClient {
  constructor(config, retryResolver) {
    this._config = config || {};
    this._retry = retryResolver || null;
    this._token = String(this._config.slackBotToken || '');
  }

  postMessage(channel, text, blocks) { return this._call('chat.postMessage', { channel: channel, t
ext: text, blocks: blocks || [] }); }
  postEphemeral(channel, user, text, blocks) { return this._call('chat.postEphemeral', { channel:
channel, user: user, text: text, blocks: blocks || [] }); }
  openView(triggerId, view) { return this._call('views.open', { trigger_id: triggerId, view: view
}); }
  updateMessage(channel, ts, text, blocks) { return this._call('chat.update', { channel: channel,
ts: ts, text: text, blocks: blocks || [] }); }
  openDm(userId) {
    var res = this._call('conversations.open', { users: userId });
    return { ok: !!res.ok, code: res.code, message: res.message, channelId: res.data && res.data.c
hannel ? res.data.channel.id : '' };
  }

  _call(method, payload) {
    if (!this._token) {
      return { ok: false, code: 'MISSING_BOT_TOKEN', message: 'SLACK_BOT_TOKEN missing', retryable
: false };
    }

    var response;
    try {
      response = UrlFetchApp.fetch('https://slack.com/api/' + method, {
        method: 'post',
        contentType: 'application/json; charset=utf-8',
        headers: { Authorization: 'Bearer ' + this._token },
        payload: JSON.stringify(payload || {}),
        muteHttpExceptions: true
      });
    } catch (err) {
      var fetchErr = { ok: false, code: 'HTTP_FETCH_FAILED', message: String(err.message || err),
retryable: true, method: method };
      this._queueRetry(method, payload, fetchErr);
      return fetchErr;
    }

    var status = response.getResponseCode();
    var body = {};
    try { body = JSON.parse(response.getContentText() || '{}'); } catch (e) { body = { ok: false,
error: 'invalid_json' }; }

    if (status < 200 || status >= 300 || !body.ok) {
      var errShape = {
        ok: false,
        code: body.error || 'SLACK_API_ERROR',
        message: 'Slack API call failed for ' + method,
        retryable: status >= 500 || body.error === 'ratelimited',
        method: method,
        status: status,
        data: body
      };
      this._queueRetry(method, payload, errShape);
      return errShape;
    }
```

```
    return { ok: true, code: 'OK', message: 'success', data: body };
  }

  _queueRetry(method, payload, errShape) {
    if (!this._retry || !errShape.retryable) return;
    this._retry.scheduleRetry({
      jobType: 'slack_api:' + method,
      payload: payload,
      lastError: errShape.code
    });
  }
}
```

FILE PATH: 12_SlackBlockKitBuilder.gs

```
class SlackBlockKitBuilder {
  buildWelcomeMessage(input) {
    return [{ type: 'section', text: { type: 'mrkdwn', text: '*Welcome to RWR LMS*\nCourse: ' + (i
nput.courseId || '') } }];
  }

  buildLessonCard(input) {
    return [{ type: 'section', text: { type: 'mrkdwn', text: '*Lesson:* ' + (input.title || input.
lessonId || '') } }];
  }

  buildProgressSummary(input) {
    return [{ type: 'section', text: { type: 'mrkdwn', text: '*Progress*\nCompleted: ' + input.com
pletedCount + '\nOverdue: ' + input.overdueCount + '\nNext: ' + input.nextAction } }];
  }

  buildReminder(input) {
    return [{ type: 'section', text: { type: 'mrkdwn', text: ':bell: Reminder for learner ' + (inp
ut.learnerId || '') } }];
  }

  buildAdminSummary(input) {
    return [{ type: 'section', text: { type: 'mrkdwn', text: '*Weekly Summary*\n' + JSON.stringify
(input || {}) } }];
  }

  buildSubmitModal(input) {
    return {
      type: 'modal',
      title: { type: 'plain_text', text: 'Submit Lesson' },
      private_metadata: JSON.stringify(input || {}),
      submit: { type: 'plain_text', text: 'Submit' },
      close: { type: 'plain_text', text: 'Cancel' },
      blocks: [{ type: 'input', block_id: 'lesson', element: { type: 'plain_text_input', action_id
: 'lesson_id' }, label: { type: 'plain_text', text: 'Lesson ID' } }]
    };
  }

  buildGenericErrorResponse(message) {
    return [{ type: 'section', text: { type: 'mrkdwn', text: ':warning: ' + (message || 'Unexpecte
d error') } }];
  }
}
```

FILE PATH: 13_LearnerProgressStateMachine.gs

```
class LearnerProgressStateMachine {
  constructor() {
    this.states = {
      QUEUED: 'queued',
      DELIVERED: 'delivered',
      STARTED: 'started',
      SUBMITTED: 'submitted',
      COMPLETED: 'completed',
      OVERDUE: 'overdue'
    };
    this._allowed = {};
    this._allowed[this.states.QUEUED] = [this.states.DELIVERED];
    this._allowed[this.states.DELIVERED] = [this.states.STARTED, this.states.OVERDUE];
    this._allowed[this.states.STARTED] = [this.states.SUBMITTED, this.states.OVERDUE];
    this._allowed[this.states.SUBMITTED] = [this.states.COMPLETED];
    this._allowed[this.states.COMPLETED] = [];
    this._allowed[this.states.OVERDUE] = [this.states.STARTED, this.states.SUBMITTED, this.states.
COMPLETED];
  }

  canTransition(fromState, toState) {
    var from = String(fromState || this.states.QUEUED);
    var to = String(toState || '');
    return (this._allowed[from] || []).indexOf(to) !== -1;
  }

  transition(record, toState, meta) {
    var source = record || {};
    var fromState = source.state || this.states.QUEUED;
    if (!this.canTransition(fromState, toState)) {
      return {
        ok: false,
        code: 'INVALID_TRANSITION',
        message: fromState + ' -> ' + toState + ' not allowed.',
        record: source,
        meta: meta || {}
      };
    }

    var next = {};
    Object.keys(source).forEach(function(k) { next[k] = source[k]; });
    next.state = toState;
    next.updatedAt = new Date().toISOString();

    return { ok: true, code: 'TRANSITION_OK', message: 'State updated', record: next, meta: meta |
| {} };
  }
}
```

FILE PATH: 14_Scheduler.gs

```
function setupTriggers() {
  removeTriggers();
  ScriptApp.newTrigger('runDailyLessonDelivery').timeBased().everyDays(1).atHour(8).create();
  ScriptApp.newTrigger('runHourlyReminderCheck').timeBased().everyHours(1).create();
  ScriptApp.newTrigger('runWeeklyAdminReport').timeBased().onWeekDay(ScriptApp.WeekDay.MONDAY).atH
our(9).create();
  return { ok: true, code: 'TRIGGERS_CREATED' };
}

function removeTriggers() {
  ScriptApp.getProjectTriggers().forEach(function(t) { ScriptApp.deleteTrigger(t); });
  return { ok: true, code: 'TRIGGERS_REMOVED' };
}

function runDailyLessonDelivery() {
  var lock = LockService.getScriptLock();
  lock.waitLock(30000);
  try {
    var deps = createHostDependencies(ConfigBootstrap.load());
    return deps.lessonService.deliverPendingLessons();
  } finally {
    lock.releaseLock();
  }
}

function runHourlyReminderCheck() {
  var lock = LockService.getScriptLock();
  lock.waitLock(30000);
  try {
    var deps = createHostDependencies(ConfigBootstrap.load());
    return deps.reminderService.sendOverdueReminders();
  } finally {
    lock.releaseLock();
  }
}

function runWeeklyAdminReport() {
  var lock = LockService.getScriptLock();
  lock.waitLock(30000);
  try {
    var deps = createHostDependencies(ConfigBootstrap.load());
    return deps.reportService.buildWeeklySummary();
  } finally {
    lock.releaseLock();
  }
}

function runHealthCheck() {
  return createHostDependencies(ConfigBootstrap.load()).healthMonitor.getSnapshot();
}
```

FILE PATH: 15_SheetsDataSync.gs

```
function syncApprovedLessonsToRuntime() {
  // TODO: map authored lesson source rows once source schema is finalized.
  return { ok: true, upserted: 0, retired: deactivateRetiredLessons().retired };
}

function upsertLessonRuntimeRecord(row) {
  var db = createHostDbClient(ConfigBootstrap.load());
  var table = db.table('lessons');
  var existing = row && row.id ? table.findById(row.id) : null;

  if (existing) {
    var updated = table.update(existing.id, row);
    return { ok: true, code: 'UPDATED', lessonId: updated.id };
  }

  var inserted = table.insert({
    id: row.id || undefined,
    courseId: row.courseId || '',
    track: row.track || '',
    title: row.title || '',
    contentRef: row.contentRef || '',
    active: String(row.active == null ? 'true' : row.active)
  });
  return { ok: true, code: 'INSERTED', lessonId: inserted.id };
}

function deactivateRetiredLessons() {
  return { ok: true, retired: 0 };
}
```

FILE PATH: 16_ConfigBootstrap.gs

```
var ConfigBootstrap = {
  load: function() {
    var helper = new ScriptPropertiesHelper();
    return {
      slackBotToken: helper.getRequired('SLACK_BOT_TOKEN'),
      slackSigningSecret: helper.getRequired('SLACK_SIGNING_SECRET'),
      spreadsheetId: helper.get('SHEET_DB_SPREADSHEET_ID', helper.getRequired('SPREADSHEET_ID')),
      slackVerificationToken: helper.get('SLACK_VERIFICATION_TOKEN', ''),
      adminUserIds: this._csv(helper.get('ADMIN_USER_IDS', '')),
      defaultCourseId: helper.get('DEFAULT_COURSE_ID', 'C001'),
      defaultTrack: helper.get('DEFAULT_TRACK', 'ONBOARDING'),
      opsAlertChannel: helper.get('OPS_ALERT_CHANNEL', '')
    };
  },

  validate: function() {
    try {
      this.load();
      return { ok: true, code: 'OK', message: 'Configuration valid.' };
    } catch (err) {
      return { ok: false, code: 'CONFIG_INVALID', message: String(err.message || err) };
    }
  },

  get: function(key, fallback) {
    var cfg = this.load();
    return cfg.hasOwnProperty(key) ? cfg[key] : fallback;
  },

  _csv: function(text) {
    return String(text || '').split(',').map(function(v) { return v.trim(); }).filter(function(v)
{ return !!v; });
  }
};
```

FILE PATH: 17_HealthMonitor.gs

```
class HealthMonitor {
  constructor(db, config) {
    this._db = db;
    this._config = config || {};
  }

  recordJobStatus(jobName, status, metadata) {
    var row = this._db.table('audit_log').insert({
      actor: 'scheduler',
      action: jobName,
      resourceType: 'job',
      resourceId: '',
      status: status,
      message: jobName + ':' + status,
      metadata: JSON.stringify(metadata || {})
    });
    return { ok: true, auditId: row.id };
  }

  recordSlackApiFailure(errorShape) {
    return this.recordJobStatus('slack_api_failure', 'error', errorShape || {});
  }

  getQueueDepthReport() {
    return {
      ok: true,
      deliveryQueue: this._db.table('delivery_queue').findAll().length,
      retryQueue: this._db.table('retry_queue').findAll().length
    };
  }

  getOverdueMetrics() {
    var overdue = this._db.table('learner_progress').findAll().filter(function(r) { return r.state
 === 'overdue'; });
    return { ok: true, overdueCount: overdue.length };
  }

  getSnapshot() {
    return {
      ok: true,
      timestamp: new Date().toISOString(),
      config: ConfigBootstrap.validate(),
      sheetDb: SheetDb.healthCheck(),
      queues: this.getQueueDepthReport(),
      overdue: this.getOverdueMetrics()
    };
  }
}
```

FILE PATH: 18_RetryResolver.gs

```
class RetryResolver {
  constructor(db, config) {
    this._db = db;
    this._config = config || {};
    this._maxAttempts = 5;
  }

  isRetryable(errorShape) {
    if (!errorShape) return false;
    if (errorShape.retryable) return true;
    return ['HTTP_FETCH_FAILED', 'ratelimited', 'internal_error'].indexOf(String(errorShape.code |
| '')) !== -1;
  }

  scheduleRetry(job) {
    var payload = job || {};
    return this._db.table('retry_queue').insert({
      jobType: payload.jobType || 'unknown',
      payload: JSON.stringify(payload.payload || {}),
      attempts: String(payload.attempts || 0),
      nextRunAt: payload.nextRunAt || new Date().toISOString(),
      status: 'queued',
      lastError: payload.lastError || ''
    });
  }

  resolveDueRetries(limit) {
    var max = Number(limit || 20);
    var due = this._db.table('retry_queue').findAll().filter(function(r) { return r.status === 'qu
eued'; }).slice(0, max);
    return { ok: true, dueCount: due.length, rows: due };
  }

  markAttempt(rowId, attempts, lastError) {
    var nextAttempts = Number(attempts || 0) + 1;
    var dead = nextAttempts >= this._maxAttempts;
    return this._db.table('retry_queue').update(rowId, {
      attempts: String(nextAttempts),
      status: dead ? 'dead_letter' : 'queued',
      lastError: lastError || '',
      nextRunAt: new Date(Date.now() + Math.pow(2, nextAttempts) * 60000).toISOString()
    });
  }
}
```

FILE PATH: 19_HostTestHarness.gs

```
function hostTest_fakeSlashLesson() {
  return doPost(_fakeSlash('/lesson', '')).getContent();
}

function hostTest_fakeSlashSubmit() {
  return doPost(_fakeSlash('/submit', 'L001')).getContent();
}

function hostTest_fakeSlashProgress() {
  return doPost(_fakeSlash('/progress', '')).getContent();
}

function hostTest_fakeInteractive() {
  var payload = {
    type: 'block_actions',
    user: { id: 'U123' },
    channel: { id: 'C123' },
    team: { id: 'T123' },
    actions: [{ action_id: 'submit_lesson' }]
  };
  return doPost({
    parameter: { payload: JSON.stringify(payload), token: 'fallback' },
    postData: { type: 'application/x-www-form-urlencoded', contents: 'payload=' + encodeURICompone
nt(JSON.stringify(payload)) }
  }).getContent();
}

function hostTest_fakeAppMention() {
  return doPost(_fakeEvent('app_mention')).getContent();
}

function hostTest_fakeMessageIm() {
  return doPost(_fakeEvent('message', 'im')).getContent();
}

function hostTest_reminderSmoke() {
  return JSON.stringify(runHourlyReminderCheck());
}

function hostTest_dailyDeliverySmoke() {
  return JSON.stringify(runDailyLessonDelivery());
}

function _fakeSlash(command, text) {
  var body = 'command=' + encodeURIComponent(command) + '&text=' + encodeURIComponent(text || '')
+ '&user_id=U123&channel_id=C123&team_id=T123&token=fallback';
  return {
    parameter: { command: command, text: text || '', user_id: 'U123', channel_id: 'C123', team_id:
 'T123', token: 'fallback' },
    postData: { type: 'application/x-www-form-urlencoded', contents: body }
  };
}

function _fakeEvent(eventType, channelType) {
  var body = {
    type: 'event_callback',
    token: 'fallback',
    team_id: 'T123',
    event: { type: eventType, channel_type: channelType || 'channel', user: 'U123', channel: 'C123
' }
  };
  return {
```

```
    parameter: {},
    postData: { type: 'application/json', contents: JSON.stringify(body) }
  };
}
```

FILE PATH: 20_Errors.gs

```
/**
 * Base error for the Sheet DB library.
 */
class SheetDbError extends Error {
  /**
   * @param {string} message Human-readable error message.
   * @param {string=} code Stable error code for programmatic handling.
   */
  constructor(message, code) {
    super(message);
    this.name = this.constructor.name;
    this.code = code || 'SHEET_DB_ERROR';
  }
}

/**
 * Raised when configuration is invalid or missing.
 */
class ConfigError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'CONFIG_ERROR');
  }
}

/**
 * Raised when schema definitions are invalid.
 */
class SchemaError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'SCHEMA_ERROR');
  }
}

/**
 * Raised when records fail validation.
 */
class ValidationError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'VALIDATION_ERROR');
  }
}

/**
 * Raised when underlying sheet operations fail.
 */
class StorageError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'STORAGE_ERROR');
  }
}
```

```
/**
 * Raised for optimistic transaction failures.
 */
class TransactionError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'TRANSACTION_ERROR');
  }
}
```

FILE PATH: 21_Util.gs

```
/**
 * Utility helpers for Sheet DB.
 */
class Util {
  /**
   * @return {string} RFC3339 timestamp in UTC.
   */
  static nowIso() {
    return new Date().toISOString();
  }

  /**
   * Creates a shallow clone of an object.
   * @param {Object} source
   * @return {Object}
   */
  static clone(source) {
    return Object.assign({}, source || {});
  }

  /**
   * Creates a stable row ID with a prefix.
   * @param {string=} prefix
   * @return {string}
   */
  static generateId(prefix) {
    var p = prefix || 'row';
    var random = Math.random().toString(36).slice(2, 10);
    return p + '_' + new Date().getTime() + '_' + random;
  }

  /**
   * Returns true when value is null or undefined.
   * @param {*} value
   * @return {boolean}
   */
  static isNil(value) {
    return value === null || value === undefined;
  }

  /**
   * Returns true when value is empty string, null, or undefined.
   * @param {*} value
   * @return {boolean}
   */
  static isBlank(value) {
    return value === '' || Util.isNil(value);
  }

  /**
   * Throws when a required condition is not met.
   * @param {boolean} condition
   * @param {string} message
   */
  static assert(condition, message) {
    if (!condition) {
      throw new ValidationError(message);
    }
  }

  /**
   * Converts a sheet row to an object using the header order.
```

```
   * @param {string[]} headers
   * @param {Array<*>} row
   * @return {Object}
   */
  static rowToObject(headers, row) {
    var out = {};
    for (var i = 0; i < headers.length; i++) {
      out[headers[i]] = row[i];
    }
    return out;
  }

  /**
   * Converts an object to row array by header order.
   * @param {string[]} headers
   * @param {Object} obj
   * @return {Array<*>}
   */
  static objectToRow(headers, obj) {
    return headers.map(function(h) {
      return obj[h];
    });
  }
}
```

FILE PATH: 22_ScriptProperties.gs

```
/**
 * Script Properties helper for secure and environment-specific configuration.
 */
class ScriptPropertiesHelper {
  /**
   * Creates a helper bound to Script Properties store.
   */
  constructor() {
    this._props = PropertiesService.getScriptProperties();
  }

  /**
   * Returns a property value, or a default when missing.
   * @param {string} key
   * @param {*=} defaultValue
   * @return {*}
   */
  get(key, defaultValue) {
    var value = this._props.getProperty(key);
    return value === null ? defaultValue : value;
  }

  /**
   * Returns a required property value or throws when missing/blank.
   * @param {string} key
   * @return {string}
   */
  getRequired(key) {
    var value = this.get(key, '');
    if (value === '') {
      throw new ConfigError('Missing required script property: ' + key);
    }
    return String(value);
  }

  /**
   * Returns a boolean property value.
   * Accepted true values: true, 1, yes, y, on.
   * Accepted false values: false, 0, no, n, off.
   * @param {string} key
   * @param {boolean=} defaultValue
   * @return {boolean}
   */
  getBoolean(key, defaultValue) {
    var raw = this.get(key, null);
    if (raw === null || raw === '') {
      return defaultValue === undefined ? false : !!defaultValue;
    }
    var normalized = String(raw).trim().toLowerCase();
    if (['true', '1', 'yes', 'y', 'on'].indexOf(normalized) !== -1) {
      return true;
    }
    if (['false', '0', 'no', 'n', 'off'].indexOf(normalized) !== -1) {
      return false;
    }
    throw new ConfigError('Script property is not a valid boolean: ' + key);
  }

  /**
   * Returns a numeric property value.
   * @param {string} key
   * @param {number=} defaultValue
```

```
 * @return {number}
 */
getNumber(key, defaultValue) {
  var raw = this.get(key, null);
  if (raw === null || raw === '') {
    return defaultValue === undefined ? 0 : Number(defaultValue);
  }
  var n = Number(raw);
  if (isNaN(n)) {
    throw new ConfigError('Script property is not a valid number: ' + key);
  }
  return n;
}

/**
 * Returns all script properties as key-value object.
 * @return {Object<string, string>}
 */
getAll() {
  return this._props.getProperties();
}

/**
 * Returns all script properties with sensitive values redacted.
 * @return {Object<string, string>}
 */
getAllRedacted() {
  var all = this.getAll();
  var out = {};
  Object.keys(all).forEach(function(key) {
    out[key] = ScriptPropertiesHelper.shouldRedactKey(key) ? '[REDACTED]' : all[key];
  });
  return out;
}

/**
 * Sets a script property value.
 * @param {string} key
 * @param {*} value
 */
set(key, value) {
  this._props.setProperty(key, String(value));
}

/**
 * Sets multiple script properties.
 * @param {Object<string, *>} obj
 */
setMany(obj) {
  var stringified = {};
  Object.keys(obj || {}).forEach(function(key) {
    stringified[key] = String(obj[key]);
  });
  this._props.setProperties(stringified, false);
}

/**
 * Deletes a script property.
 * @param {string} key
 */
delete(key) {
  this._props.deleteProperty(key);
}
```

```
  /**
   * Returns true when a script property exists and is non-empty.
   * @param {string} key
   * @return {boolean}
   */
  has(key) {
    var value = this._props.getProperty(key);
    return value !== null && value !== '';
  }

  /**
   * Returns true when key should be redacted in debug output.
   * @param {string} key
   * @return {boolean}
   */
  static shouldRedactKey(key) {
    var upper = String(key || '').toUpperCase();
    if (upper === 'APP_PASSCODE' || upper === 'SIGNING_SECRET' || upper === 'WEBHOOK_SECRET' || up
per === 'API_KEY') {
      return true;
    }
    return upper.indexOf('TOKEN') !== -1 || upper.indexOf('SECRET') !== -1 || upper.indexOf('PASSW
ORD') !== -1;
  }
}

/**
 * Reads SheetDb client-related config from Script Properties.
 *
 * Supported keys:
 * - SHEET_DB_SPREADSHEET_ID
 * - SHEET_DB_ID_COLUMN
 * - SHEET_DB_CREATED_AT_COLUMN
 * - SHEET_DB_UPDATED_AT_COLUMN
 * - SHEET_DB_DELETED_AT_COLUMN
 * - SHEET_DB_LOCK_TIMEOUT_MS
 *
 * Also reserved for host-layer secure use:
 * - APP_PASSCODE
 * - SIGNING_SECRET
 * - WEBHOOK_SECRET
 * - API_KEY
 *
 * @return {Object}
 */
function readSheetDbConfigFromScriptProperties() {
  var helper = new ScriptPropertiesHelper();
  var config = {};

  if (helper.has('SHEET_DB_SPREADSHEET_ID')) {
    config.spreadsheetId = helper.get('SHEET_DB_SPREADSHEET_ID', '');
  }
  if (helper.has('SHEET_DB_ID_COLUMN')) {
    config.idColumn = helper.get('SHEET_DB_ID_COLUMN', 'id');
  }
  if (helper.has('SHEET_DB_CREATED_AT_COLUMN')) {
    config.createdAtColumn = helper.get('SHEET_DB_CREATED_AT_COLUMN', 'createdAt');
  }
  if (helper.has('SHEET_DB_UPDATED_AT_COLUMN')) {
    config.updatedAtColumn = helper.get('SHEET_DB_UPDATED_AT_COLUMN', 'updatedAt');
  }
  if (helper.has('SHEET_DB_DELETED_AT_COLUMN')) {
    config.deletedAtColumn = helper.get('SHEET_DB_DELETED_AT_COLUMN', 'deletedAt');
  }
```

```
  if (helper.has('SHEET_DB_LOCK_TIMEOUT_MS')) {
    config.lockTimeoutMs = helper.getNumber('SHEET_DB_LOCK_TIMEOUT_MS', 30000);
  }

  return config;
}

/**
 * Merges explicit client options with script properties-based config.
 * Precedence: explicit options > script properties > hardcoded defaults.
 *
 * @param {Object=} options
 * @return {Object}
 */
function mergeClientOptionsWithScriptProperties(options) {
  var fromProps = readSheetDbConfigFromScriptProperties();
  var explicit = options || {};
  var merged = {};
  Object.keys(fromProps).forEach(function(key) {
    merged[key] = fromProps[key];
  });
  Object.keys(explicit).forEach(function(key) {
    merged[key] = explicit[key];
  });
  return merged;
}

/**
 * Example setup helper for consumers (manual one-time setup).
 *
 * Example:
 * PropertiesService.getScriptProperties().setProperty('SHEET_DB_SPREADSHEET_ID', '...');
 */
function exampleSetSheetDbScriptProperties() {
  // Intentionally no-op; serves as in-library usage documentation only.
}
```

FILE PATH: 23_Config.gs

```
/**
 * Immutable runtime configuration for the Sheet DB client.
 */
class Config {
  /**
   * @param {Object=} options
   * @param {string=} options.spreadsheetId Spreadsheet ID; defaults to active spreadsheet.
   * @param {string=} options.idColumn Record primary key column name.
   * @param {string=} options.createdAtColumn Created timestamp column name.
   * @param {string=} options.updatedAtColumn Updated timestamp column name.
   * @param {string=} options.deletedAtColumn Soft-delete timestamp column name.
   * @param {number=} options.lockTimeoutMs Lock acquisition timeout for transactions.
   */
  constructor(options) {
    var opts = options || {};
    this.spreadsheetId = opts.spreadsheetId || '';
    this.idColumn = opts.idColumn || 'id';
    this.createdAtColumn = opts.createdAtColumn || 'createdAt';
    this.updatedAtColumn = opts.updatedAtColumn || 'updatedAt';
    this.deletedAtColumn = opts.deletedAtColumn || 'deletedAt';
    this.lockTimeoutMs = Util.isNil(opts.lockTimeoutMs) ? 30000 : Number(opts.lockTimeoutMs);
    this.validate();
    Object.freeze(this);
  }

  /**
   * @return {void}
   */
  validate() {
    var columns = [
      this.idColumn,
      this.createdAtColumn,
      this.updatedAtColumn,
      this.deletedAtColumn
    ];
    var unique = {};
    columns.forEach(function(column) {
      if (!column || typeof column !== 'string') {
        throw new ConfigError('System column names must be non-empty strings.');
      }
      if (unique[column]) {
        throw new ConfigError('System column names must be unique. Duplicate: ' + column);
      }
      unique[column] = true;
    });

    if (isNaN(this.lockTimeoutMs) || this.lockTimeoutMs <= 0) {
      throw new ConfigError('lockTimeoutMs must be a positive number.');
    }
  }
}
```

FILE PATH: 24_SheetsGateway.gs

```
/**
 * Thin wrapper around SpreadsheetApp APIs.
 */
class SheetsGateway {
  /**
   * @param {Config} config
   */
  constructor(config) {
    this._config = config;
  }

  /**
   * @return {GoogleAppsScript.Spreadsheet.Spreadsheet}
   */
  getSpreadsheet() {
    if (this._config.spreadsheetId) {
      return SpreadsheetApp.openById(this._config.spreadsheetId);
    }
    return SpreadsheetApp.getActiveSpreadsheet();
  }

  /**
   * @param {string} tableName
   * @param {boolean=} createWhenMissing
   * @return {GoogleAppsScript.Spreadsheet.Sheet}
   */
  getSheet(tableName, createWhenMissing) {
    var ss = this.getSpreadsheet();
    var sheet = ss.getSheetByName(tableName);
    if (!sheet && createWhenMissing) {
      sheet = ss.insertSheet(tableName);
    }
    if (!sheet) {
      throw new StorageError('Sheet not found for table: ' + tableName);
    }
    return sheet;
  }

  /**
   * @param {string} tableName
   * @return {{headers: string[], rows: Array<Array<*>>}}
   */
  readTable(tableName) {
    var sheet = this.getSheet(tableName, false);
    var range = sheet.getDataRange();
    var values = range.getValues();
    if (values.length === 0) {
      return { headers: [], rows: [] };
    }
    return {
      headers: values[0],
      rows: values.slice(1)
    };
  }

  /**
   * Ensures table exists with given headers.
   * @param {string} tableName
   * @param {string[]} headers
   */
  ensureTable(tableName, headers) {
    var sheet = this.getSheet(tableName, true);
```

```
    var lastRow = sheet.getLastRow();
    if (lastRow === 0) {
      sheet.getRange(1, 1, 1, headers.length).setValues([headers]);
      return;
    }

    var existingHeaders = sheet.getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0]
      .map(function(value) { return String(value); });
    var expectedHeaders = headers.map(function(value) { return String(value); });
    if (existingHeaders.join('|') !== expectedHeaders.join('|')) {
      throw new StorageError('Header mismatch for table: ' + tableName);
    }
  }

  /**
   * Writes all rows for a table, replacing existing body rows.
   * @param {string} tableName
   * @param {string[]} headers
   * @param {Array<Array<*>>} rows
   */
  writeAllRows(tableName, headers, rows) {
    var sheet = this.getSheet(tableName, true);
    this.ensureTable(tableName, headers);

    var maxRows = Math.max(sheet.getMaxRows(), rows.length + 1);
    if (sheet.getMaxRows() < maxRows) {
      sheet.insertRowsAfter(sheet.getMaxRows(), maxRows - sheet.getMaxRows());
    }

    if (sheet.getLastRow() > 1) {
      sheet.getRange(2, 1, sheet.getLastRow() - 1, headers.length).clearContent();
    }

    if (rows.length > 0) {
      sheet.getRange(2, 1, rows.length, headers.length).setValues(rows);
    }
  }
}
```

FILE PATH: 25_SchemaRegistry.gs

```
/**
 * Holds table schemas and applies field defaults/validation.
 */
class SchemaRegistry {
  /**
   * Creates a new in-memory schema registry.
   */
  constructor() {
    this._schemas = {};
  }

  /**
   * Registers a table schema.
   * @param {string} tableName
   * @param {Object} schema
   * @param {Array<string>} schema.columns Ordered columns for persistence.
   * @param {Object<string, Object>=} schema.fields Optional field constraints.
   * @return {SchemaRegistry}
   */
  register(tableName, schema) {
    if (!tableName) {
      throw new SchemaError('Table name is required.');
    }
    if (!schema || !Array.isArray(schema.columns) || schema.columns.length === 0) {
      throw new SchemaError('Schema for ' + tableName + ' must provide non-empty columns array.');
    }
    var seen = {};
    schema.columns.forEach(function(column) {
      if (!column || typeof column !== 'string') {
        throw new SchemaError('All columns must be non-empty strings for table: ' + tableName);
      }
      if (seen[column]) {
        throw new SchemaError('Duplicate column "' + column + '" in table: ' + tableName);
      }
      seen[column] = true;
    });

    var fields = Util.clone(schema.fields || {});
    Object.keys(fields).forEach(function(fieldName) {
      if (!seen[fieldName]) {
        throw new SchemaError(
          'Field rules defined for unknown column "' + fieldName + '" in table: ' + tableName
        );
      }
    });

    this._schemas[tableName] = {
      columns: schema.columns.slice(),
      fields: fields
    };
    return this;
  }

  /**
   * Returns schema for a table.
   * @param {string} tableName
   * @return {Object}
   */
  get(tableName) {
    var schema = this._schemas[tableName];
    if (!schema) {
      throw new SchemaError('No schema registered for table: ' + tableName);
```

```
    }
    return {
      columns: schema.columns.slice(),
      fields: Util.clone(schema.fields)
    };
  }

  /**
   * Validates and normalizes a record.
   * @param {string} tableName
   * @param {Object} record
   * @param {boolean=} isPartial True for patch-like updates.
   * @return {Object}
   */
  normalizeRecord(tableName, record, isPartial) {
    var schema = this.get(tableName);
    var out = Util.clone(record || {});
    schema.columns.forEach(function(column) {
      var fieldRules = schema.fields[column] || {};
      var hasValue = !Util.isNil(out[column]);

      if (!hasValue && fieldRules.hasOwnProperty('default')) {
        var defaultValue = fieldRules.default;
        out[column] = typeof defaultValue === 'function' ? defaultValue() : defaultValue;
      }

      if (!isPartial && fieldRules.required && Util.isNil(out[column])) {
        throw new ValidationError('Field "' + column + '" is required.');
      }

      if (!Util.isNil(out[column]) && fieldRules.type) {
        var expected = fieldRules.type;
        var actual = Object.prototype.toString.call(out[column]).slice(8, -1).toLowerCase();
        if (actual !== expected.toLowerCase()) {
          throw new ValidationError(
            'Field "' + column + '" expected type ' + expected + ' but got ' + actual + '.'
          );
        }
      }
    });

    return out;
  }
}
```

FILE PATH: 26_TableRepository.gs

```
/**
 * CRUD repository for a specific table.
 */
class TableRepository {
  /**
   * @param {string} tableName
   * @param {Config} config
   * @param {SchemaRegistry} schemaRegistry
   * @param {SheetsGateway} sheetsGateway
   * @param {AuditLogger} auditLogger
   */
  constructor(tableName, config, schemaRegistry, sheetsGateway, auditLogger) {
    this._tableName = tableName;
    this._config = config;
    this._schemas = schemaRegistry;
    this._sheets = sheetsGateway;
    this._audit = auditLogger;
  }

  /**
   * @return {Array<Object>}
   */
  findAll() {
    var schema = this._schemas.get(this._tableName);
    this._sheets.ensureTable(this._tableName, schema.columns);
    var data = this._sheets.readTable(this._tableName);
    return data.rows
      .map(function(row) { return Util.rowToObject(data.headers, row); })
      .filter(function(record) { return Util.isBlank(record[this._config.deletedAtColumn]); }, thi
s);
  }

  /**
   * @param {string} id
   * @return {Object|null}
   */
  findById(id) {
    var rows = this.findAll();
    for (var i = 0; i < rows.length; i++) {
      if (rows[i][this._config.idColumn] === id) {
        return rows[i];
      }
    }
    return null;
  }

  /**
   * @param {Object} payload
   * @return {Object}
   */
  insert(payload) {
    var schema = this._schemas.get(this._tableName);
    this._sheets.ensureTable(this._tableName, schema.columns);
    var record = this._schemas.normalizeRecord(this._tableName, payload, false);
    var now = Util.nowIso();
    record[this._config.idColumn] = record[this._config.idColumn] || Util.generateId(this._tableNa
me);
    record[this._config.createdAtColumn] = record[this._config.createdAtColumn] || now;
    record[this._config.updatedAtColumn] = now;
    if (!record.hasOwnProperty(this._config.deletedAtColumn)) {
      record[this._config.deletedAtColumn] = '';
    }
```

```
    var tableData = this._sheets.readTable(this._tableName);
    var existingRows = tableData.rows.map(function(row) { return Util.rowToObject(tableData.header
s, row); });
    var idColumn = this._config.idColumn;
    var duplicate = existingRows.some(function(row) {
      return row[idColumn] === record[idColumn];
    });
    if (duplicate) {
      throw new ValidationError('Record ID already exists: ' + record[idColumn]);
    }
    existingRows.push(record);
    var rowValues = existingRows.map(function(r) { return Util.objectToRow(schema.columns, r); });
    this._sheets.writeAllRows(this._tableName, schema.columns, rowValues);
    this._audit.log('insert', this._tableName, { id: record[this._config.idColumn] });
    return record;
  }

  /**
   * @param {string} id
   * @param {Object} patch
   * @return {Object}
   */
  update(id, patch) {
    var schema = this._schemas.get(this._tableName);
    var normalizedPatch = this._schemas.normalizeRecord(this._tableName, patch, true);
    var tableData = this._sheets.readTable(this._tableName);
    var records = tableData.rows.map(function(row) { return Util.rowToObject(tableData.headers, ro
w); });
    var found = null;

    for (var i = 0; i < records.length; i++) {
      if (records[i][this._config.idColumn] === id) {
        found = records[i];
        Object.keys(normalizedPatch).forEach(function(key) {
          if (key !== this._config.idColumn && key !== this._config.createdAtColumn) {
            found[key] = normalizedPatch[key];
          }
        }, this);
        found[this._config.updatedAtColumn] = Util.nowIso();
        break;
      }
    }

    if (!found) {
      throw new StorageError('Record not found for id: ' + id);
    }

    var rowValues = records.map(function(r) { return Util.objectToRow(schema.columns, r); });
    this._sheets.writeAllRows(this._tableName, schema.columns, rowValues);
    this._audit.log('update', this._tableName, { id: id, fields: Object.keys(normalizedPatch) });
    return found;
  }

  /**
   * Soft-deletes a record.
   * @param {string} id
   * @return {Object}
   */
  remove(id) {
    var result = this.update(id, (function(col, now) {
      var patch = {};
      patch[col] = now;
      return patch;
```

```
    })(this._config.deletedAtColumn, Util.nowIso()));
    this._audit.log('remove', this._tableName, { id: id });
    return result;
  }
}
```

FILE PATH: 27_TransactionManager.gs

```
/**
 * Runs operations under ScriptLock for coarse transaction semantics.
 */
class TransactionManager {
  /**
   * @param {number=} lockTimeoutMs
   */
  constructor(lockTimeoutMs) {
    this._lockTimeoutMs = lockTimeoutMs || 30000;
  }

  /**
   * Executes callback while holding a script lock.
   * @template T
   * @param {function(): T} callback
   * @return {T}
   */
  run(callback) {
    var lock = LockService.getScriptLock();
    if (!lock.tryLock(this._lockTimeoutMs)) {
      throw new TransactionError('Failed to acquire script lock within timeout.');
    }
    try {
      return callback();
    } finally {
      lock.releaseLock();
    }
  }
}
```

FILE PATH: 28_AuditLogger.gs

```
/**
 * Minimal pluggable audit logger.
 */
class AuditLogger {
  /**
   * @param {Object=} options
   * @param {Function=} options.sink Optional custom sink function.
   */
  constructor(options) {
    var opts = options || {};
    this._sink = opts.sink || function(entry) {
      Logger.log(JSON.stringify(entry));
    };
  }

  /**
   * Emits an audit entry.
   * @param {string} action
   * @param {string} tableName
   * @param {Object=} metadata
   */
  log(action, tableName, metadata) {
    this._sink({
      at: Util.nowIso(),
      action: action,
      table: tableName,
      metadata: Util.clone(metadata || {})
    });
  }
}
```

FILE PATH: 29_DbClient.gs

```
/**
 * Main entrypoint for interacting with sheet-backed tables.
 */
class DbClient {
  /**
   * @param {Config=} config
   */
  constructor(config) {
    this._config = config || new Config();
    this._schemas = new SchemaRegistry();
    this._sheets = new SheetsGateway(this._config);
    this._audit = new AuditLogger();
    this._tx = new TransactionManager(this._config.lockTimeoutMs);
  }

  /**
   * Registers a table schema.
   * @param {string} tableName
   * @param {Object} schema
   * @return {DbClient}
   */
  registerTable(tableName, schema) {
    this._schemas.register(tableName, schema);
    this._sheets.ensureTable(tableName, schema.columns);
    return this;
  }

  /**
   * Returns a table repository.
   * @param {string} tableName
   * @return {TableRepository}
   */
  table(tableName) {
    this._schemas.get(tableName);
    return new TableRepository(
      tableName,
      this._config,
      this._schemas,
      this._sheets,
      this._audit
    );
  }

  /**
   * Gets or registers a table schema.
   *
   * - When ‘schema‘ is provided, registers/overwrites the schema and ensures the table.
   * - When ‘schema‘ is omitted, returns the current schema.
   *
   * @param {string} tableName
   * @param {Object=} schema
   * @return {Object|DbClient}
   */
  schema(tableName, schema) {
    if (schema) {
      this.registerTable(tableName, schema);
      return this;
    }
    return this._schemas.get(tableName);
  }

  /**
```

```
   * Writes an audit entry through the configured audit logger.
   * @param {string} action
   * @param {string} tableName
   * @param {Object=} metadata
   */
  audit(action, tableName, metadata) {
    this._audit.log(action, tableName, metadata || {});
  }

  /**
   * Runs a set of operations under lock.
   * @template T
   * @param {function(DbClient): T} callback
   * @return {T}
   */
  transaction(callback) {
    var self = this;
    return this._tx.run(function() {
      return callback(self);
    });
  }
}
```

FILE PATH: 30_SheetDb.gs

```
/**
 * @fileoverview Public, library-visible exports for SheetDb.
 */

/**
 * Library semantic version.
 *
 * NOTE: Keep this in sync with release tags when publishing as an Apps Script library.
 * @type {string}
 */
var SHEET_DB_VERSION = '0.1.0';

/**
 * Creates a new SheetDb client instance.
 *
 * @param {Object=} options Client configuration options.
 * @return {DbClient}
 */
function _sheetDbCreateClient(options) {
  return new DbClient(new Config(options || {}));
}

/**
 * Bootstraps a client with one or more table schemas.
 *
 * @param {Object=} options
 * @param {Array<{name: string, schema: Object}>=} options.tables Tables to register.
 * @param {Object=} options.clientOptions Configuration passed to `createClient`.
 * @return {DbClient}
 */
function _sheetDbBootstrap(options) {
  var opts = options || {};
  var client = _sheetDbCreateClient(opts.clientOptions || {});
  var tables = opts.tables || [];

  tables.forEach(function(table) {
    if (table && table.name && table.schema) {
      client.registerTable(table.name, table.schema);
    }
  });

  return client;
}

/**
 * Basic health check that can be called by consuming projects.
 *
 * @return {{ok: boolean, version: string, timestamp: string, details: Object}}
 */
function _sheetDbHealthCheck() {
  try {
    var client = _sheetDbCreateClient();
    return {
      ok: !!client,
      version: SHEET_DB_VERSION,
      timestamp: Util.nowIso(),
      details: {
        clientReady: true,
        spreadsheetMode: client._config && client._config.spreadsheetId ? 'by_id' : 'active'
      }
    };
  } catch (err) {
```

```
    return {
      ok: false,
      version: SHEET_DB_VERSION,
      timestamp: new Date().toISOString(),
      details: {
        code: err.code || 'HEALTH_CHECK_FAILED',
        message: err.message || 'Unknown error.'
      }
    };
  }
}

/**
 * Creates a client using Script Properties with optional explicit overrides.
 * Precedence: explicit options > script properties > defaults.
 * @param {Object=} options
 * @return {DbClient}
 */
function _sheetDbCreateClientFromScriptProperties(options) {
  var merged = mergeClientOptionsWithScriptProperties(options || {});
  return _sheetDbCreateClient(merged);
}

/**
 * Public Apps Script library namespace.
 *
 * Consumers should call library exports through this object:
 * `LibraryIdentifier.SheetDb.createClient(...)`.
 *
 * @type {{
 *   createClient: function(Object=): DbClient,
 *   createClientFromScriptProperties: function(Object=): DbClient,
 *   bootstrap: function(Object=): DbClient,
 *   healthCheck: function(): Object,
 *   version: string
 * }}
 */
var SheetDb = {
  createClient: function(options) {
    return _sheetDbCreateClient(options);
  },

  createClientFromScriptProperties: function(options) {
    return _sheetDbCreateClientFromScriptProperties(options);
  },

  bootstrap: function(options) {
    return _sheetDbBootstrap(options);
  },

  healthCheck: function() {
    return _sheetDbHealthCheck();
  },

  version: SHEET_DB_VERSION
};

/**
 * Backward-compatible factory export.
 * @deprecated Use `SheetDb.createClient` instead.
 * @param {Object=} options
 * @return {DbClient}
 */
function createSheetDbClient(options) {
```

```
  return SheetDb.createClient(options);
}
```

FILE PATH: code.gs

```
/**************************************************
 * RWR LMS â   Slack Web App Anchor (thin host shell)
 **************************************************/

const CONFIG = Object.freeze({
  ACTOR_SYSTEM: 'system'
});

function doGet() {
  return textResponse_('alive');
}

function doPost(e) {
  try {
    var config = getHostConfig_();
    var deps = getHostDependencies_(config);
    var parsed = SlackPayloadParser.parse(e);
    var verified = SlackSecurity.verify(parsed, config);

    if (!verified.ok) {
      deps.audit('request_denied', {
        code: verified.code,
        routeType: parsed.routeType
      });
      return jsonResponse_({ ok: false, error: verified.code, message: verified.message });
    }

    var routed = SlackRouter.route(parsed, deps);
    deps.audit('request_routed', {
      routeType: parsed.routeType,
      ok: !!routed.ok,
      code: routed.code || ''
    });

    return jsonResponse_(routed.response || { ok: false, error: 'missing_response' });
  } catch (err) {
    try {
      logAuditExact_({
        actor: CONFIG.ACTOR_SYSTEM,
        action: 'doPost_error',
        status: 'error',
        message: safeError_(err)
      });
    } catch (logErr) {}

    return jsonResponse_({ ok: false, error: 'host_error', message: 'Something went wrong in the S
lack handler.' });
  }
}

function getHostConfig_() {
  return ConfigBootstrap.load();
}

function getHostDependencies_(config) {
  return createHostDependencies(config);
}

function textResponse_(text) {
  return ContentService.createTextOutput(String(text || '')).setMimeType(ContentService.MimeType.T
EXT);
}
```

```
function jsonResponse_(obj) {
  return ContentService.createTextOutput(JSON.stringify(obj || {})).setMimeType(ContentService.Mim
eType.JSON);
}

function logAuditExact_(entry) {
  Logger.log(JSON.stringify(sanitizeObjectForLog_(entry || {})));
}

function safeError_(err) {
  return (err && err.message) ? String(err.message) : String(err);
}

function sanitizeObjectForLog_(obj) {
  var clean = {};
  Object.keys(obj || {}).forEach(function(key) {
    var lower = String(key).toLowerCase();
    if (lower.indexOf('token') !== -1 || lower.indexOf('secret') !== -1 || lower.indexOf('authoriz
ation') !== -1) {
      clean[key] = '[REDACTED]';
      return;
    }
    clean[key] = String(obj[key] == null ? '' : obj[key]);
  });
  return clean;
}
```

FILE PATH: sheet_db_lib/00_SheetDb.gs

```
/**
 * @fileoverview Public, library-visible exports for SheetDb.
 */

/**
 * Library semantic version.
 *
 * NOTE: Keep this in sync with release tags when publishing as an Apps Script library.
 * @type {string}
 */
var SHEET_DB_VERSION = '0.1.0';

/**
 * Creates a new SheetDb client instance.
 *
 * @param {Object=} options Client configuration options.
 * @return {DbClient}
 */
function _sheetDbCreateClient(options) {
  return new DbClient(new Config(options || {}));
}

/**
 * Bootstraps a client with one or more table schemas.
 *
 * @param {Object=} options
 * @param {Array<{name: string, schema: Object}>=} options.tables Tables to register.
 * @param {Object=} options.clientOptions Configuration passed to 'createClient'.
 * @return {DbClient}
 */
function _sheetDbBootstrap(options) {
  var opts = options || {};
  var client = _sheetDbCreateClient(opts.clientOptions || {});
  var tables = opts.tables || [];

  tables.forEach(function(table) {
    if (table && table.name && table.schema) {
      client.registerTable(table.name, table.schema);
    }
  });

  return client;
}

/**
 * Basic health check that can be called by consuming projects.
 *
 * @return {{ok: boolean, version: string, timestamp: string, details: Object}}
 */
function _sheetDbHealthCheck() {
  try {
    var client = _sheetDbCreateClient();
    return {
      ok: !!client,
      version: SHEET_DB_VERSION,
      timestamp: Util.nowIso(),
      details: {
        clientReady: true,
        spreadsheetMode: client._config && client._config.spreadsheetId ? 'by_id' : 'active'
      }
    };
  } catch (err) {
```

```
    return {
      ok: false,
      version: SHEET_DB_VERSION,
      timestamp: new Date().toISOString(),
      details: {
        code: err.code || 'HEALTH_CHECK_FAILED',
        message: err.message || 'Unknown error.'
      }
    };
  }
}

/**
 * Creates a client using Script Properties with optional explicit overrides.
 * Precedence: explicit options > script properties > defaults.
 * @param {Object=} options
 * @return {DbClient}
 */
function _sheetDbCreateClientFromScriptProperties(options) {
  var merged = mergeClientOptionsWithScriptProperties(options || {});
  return _sheetDbCreateClient(merged);
}

/**
 * Public Apps Script library namespace.
 *
 * Consumers should call library exports through this object:
 * `LibraryIdentifier.SheetDb.createClient(...)`.
 *
 * @type {{
 *   createClient: function(Object=): DbClient,
 *   createClientFromScriptProperties: function(Object=): DbClient,
 *   bootstrap: function(Object=): DbClient,
 *   healthCheck: function(): Object,
 *   version: string
 * }}
 */
var SheetDb = {
  createClient: function(options) {
    return _sheetDbCreateClient(options);
  },

  createClientFromScriptProperties: function(options) {
    return _sheetDbCreateClientFromScriptProperties(options);
  },

  bootstrap: function(options) {
    return _sheetDbBootstrap(options);
  },

  healthCheck: function() {
    return _sheetDbHealthCheck();
  },

  version: SHEET_DB_VERSION
};

/**
 * Backward-compatible factory export.
 * @deprecated Use `SheetDb.createClient` instead.
 * @param {Object=} options
 * @return {DbClient}
 */
function createSheetDbClient(options) {
```

```
  return SheetDb.createClient(options);
}
```

FILE PATH: sheet_db_lib/01_DbClient.gs

```
/**
 * Main entrypoint for interacting with sheet-backed tables.
 */
class DbClient {
  /**
   * @param {Config=} config
   */
  constructor(config) {
    this._config = config || new Config();
    this._schemas = new SchemaRegistry();
    this._sheets = new SheetsGateway(this._config);
    this._audit = new AuditLogger();
    this._tx = new TransactionManager(this._config.lockTimeoutMs);
  }

  /**
   * Registers a table schema.
   * @param {string} tableName
   * @param {Object} schema
   * @return {DbClient}
   */
  registerTable(tableName, schema) {
    this._schemas.register(tableName, schema);
    this._sheets.ensureTable(tableName, schema.columns);
    return this;
  }

  /**
   * Returns a table repository.
   * @param {string} tableName
   * @return {TableRepository}
   */
  table(tableName) {
    this._schemas.get(tableName);
    return new TableRepository(
      tableName,
      this._config,
      this._schemas,
      this._sheets,
      this._audit
    );
  }

  /**
   * Gets or registers a table schema.
   *
   * - When ‘schema‘ is provided, registers/overwrites the schema and ensures the table.
   * - When ‘schema‘ is omitted, returns the current schema.
   *
   * @param {string} tableName
   * @param {Object=} schema
   * @return {Object|DbClient}
   */
  schema(tableName, schema) {
    if (schema) {
      this.registerTable(tableName, schema);
      return this;
    }
    return this._schemas.get(tableName);
  }

  /**
```

```
   * Writes an audit entry through the configured audit logger.
   * @param {string} action
   * @param {string} tableName
   * @param {Object=} metadata
   */
  audit(action, tableName, metadata) {
    this._audit.log(action, tableName, metadata || {});
  }

  /**
   * Runs a set of operations under lock.
   * @template T
   * @param {function(DbClient): T} callback
   * @return {T}
   */
  transaction(callback) {
    var self = this;
    return this._tx.run(function() {
      return callback(self);
    });
  }
}
```

FILE PATH: sheet_db_lib/02_Config.gs

```
/**
 * Immutable runtime configuration for the Sheet DB client.
 */
class Config {
  /**
   * @param {Object=} options
   * @param {string=} options.spreadsheetId Spreadsheet ID; defaults to active spreadsheet.
   * @param {string=} options.idColumn Record primary key column name.
   * @param {string=} options.createdAtColumn Created timestamp column name.
   * @param {string=} options.updatedAtColumn Updated timestamp column name.
   * @param {string=} options.deletedAtColumn Soft-delete timestamp column name.
   * @param {number=} options.lockTimeoutMs Lock acquisition timeout for transactions.
   */
  constructor(options) {
    var opts = options || {};
    this.spreadsheetId = opts.spreadsheetId || '';
    this.idColumn = opts.idColumn || 'id';
    this.createdAtColumn = opts.createdAtColumn || 'createdAt';
    this.updatedAtColumn = opts.updatedAtColumn || 'updatedAt';
    this.deletedAtColumn = opts.deletedAtColumn || 'deletedAt';
    this.lockTimeoutMs = Util.isNil(opts.lockTimeoutMs) ? 30000 : Number(opts.lockTimeoutMs);
    this.validate();
    Object.freeze(this);
  }

  /**
   * @return {void}
   */
  validate() {
    var columns = [
      this.idColumn,
      this.createdAtColumn,
      this.updatedAtColumn,
      this.deletedAtColumn
    ];
    var unique = {};
    columns.forEach(function(column) {
      if (!column || typeof column !== 'string') {
        throw new ConfigError('System column names must be non-empty strings.');
      }
      if (unique[column]) {
        throw new ConfigError('System column names must be unique. Duplicate: ' + column);
      }
      unique[column] = true;
    });

    if (isNaN(this.lockTimeoutMs) || this.lockTimeoutMs <= 0) {
      throw new ConfigError('lockTimeoutMs must be a positive number.');
    }
  }
}
```

FILE PATH: sheet_db_lib/03_Errors.gs

```
/**
 * Base error for the Sheet DB library.
 */
class SheetDbError extends Error {
  /**
   * @param {string} message Human-readable error message.
   * @param {string=} code Stable error code for programmatic handling.
   */
  constructor(message, code) {
    super(message);
    this.name = this.constructor.name;
    this.code = code || 'SHEET_DB_ERROR';
  }
}

/**
 * Raised when configuration is invalid or missing.
 */
class ConfigError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'CONFIG_ERROR');
  }
}

/**
 * Raised when schema definitions are invalid.
 */
class SchemaError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'SCHEMA_ERROR');
  }
}

/**
 * Raised when records fail validation.
 */
class ValidationError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'VALIDATION_ERROR');
  }
}

/**
 * Raised when underlying sheet operations fail.
 */
class StorageError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'STORAGE_ERROR');
  }
}
```

```
/**
 * Raised for optimistic transaction failures.
 */
class TransactionError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'TRANSACTION_ERROR');
  }
}
```

FILE PATH: sheet_db_lib/04_Util.gs

```
/**
 * Utility helpers for Sheet DB.
 */
class Util {
  /**
   * @return {string} RFC3339 timestamp in UTC.
   */
  static nowIso() {
    return new Date().toISOString();
  }

  /**
   * Creates a shallow clone of an object.
   * @param {Object} source
   * @return {Object}
   */
  static clone(source) {
    return Object.assign({}, source || {});
  }

  /**
   * Creates a stable row ID with a prefix.
   * @param {string=} prefix
   * @return {string}
   */
  static generateId(prefix) {
    var p = prefix || 'row';
    var random = Math.random().toString(36).slice(2, 10);
    return p + '_' + new Date().getTime() + '_' + random;
  }

  /**
   * Returns true when value is null or undefined.
   * @param {*} value
   * @return {boolean}
   */
  static isNil(value) {
    return value === null || value === undefined;
  }

  /**
   * Returns true when value is empty string, null, or undefined.
   * @param {*} value
   * @return {boolean}
   */
  static isBlank(value) {
    return value === '' || Util.isNil(value);
  }

  /**
   * Throws when a required condition is not met.
   * @param {boolean} condition
   * @param {string} message
   */
  static assert(condition, message) {
    if (!condition) {
      throw new ValidationError(message);
    }
  }

  /**
   * Converts a sheet row to an object using the header order.
```

```
   * @param {string[]} headers
   * @param {Array<*>} row
   * @return {Object}
   */
  static rowToObject(headers, row) {
    var out = {};
    for (var i = 0; i < headers.length; i++) {
      out[headers[i]] = row[i];
    }
    return out;
  }

  /**
   * Converts an object to row array by header order.
   * @param {string[]} headers
   * @param {Object} obj
   * @return {Array<*>}
   */
  static objectToRow(headers, obj) {
    return headers.map(function(h) {
      return obj[h];
    });
  }
}
```

FILE PATH: sheet_db_lib/05_SchemaRegistry.gs

```
/**
 * Holds table schemas and applies field defaults/validation.
 */
class SchemaRegistry {
  /**
   * Creates a new in-memory schema registry.
   */
  constructor() {
    this._schemas = {};
  }

  /**
   * Registers a table schema.
   * @param {string} tableName
   * @param {Object} schema
   * @param {Array<string>} schema.columns Ordered columns for persistence.
   * @param {Object<string, Object>=} schema.fields Optional field constraints.
   * @return {SchemaRegistry}
   */
  register(tableName, schema) {
    if (!tableName) {
      throw new SchemaError('Table name is required.');
    }
    if (!schema || !Array.isArray(schema.columns) || schema.columns.length === 0) {
      throw new SchemaError('Schema for ' + tableName + ' must provide non-empty columns array.');
    }
    var seen = {};
    schema.columns.forEach(function(column) {
      if (!column || typeof column !== 'string') {
        throw new SchemaError('All columns must be non-empty strings for table: ' + tableName);
      }
      if (seen[column]) {
        throw new SchemaError('Duplicate column "' + column + '" in table: ' + tableName);
      }
      seen[column] = true;
    });

    var fields = Util.clone(schema.fields || {});
    Object.keys(fields).forEach(function(fieldName) {
      if (!seen[fieldName]) {
        throw new SchemaError(
          'Field rules defined for unknown column "' + fieldName + '" in table: ' + tableName
        );
      }
    });

    this._schemas[tableName] = {
      columns: schema.columns.slice(),
      fields: fields
    };
    return this;
  }

  /**
   * Returns schema for a table.
   * @param {string} tableName
   * @return {Object}
   */
  get(tableName) {
    var schema = this._schemas[tableName];
    if (!schema) {
      throw new SchemaError('No schema registered for table: ' + tableName);
```

```
    }
    return {
      columns: schema.columns.slice(),
      fields: Util.clone(schema.fields)
    };
  }

  /**
   * Validates and normalizes a record.
   * @param {string} tableName
   * @param {Object} record
   * @param {boolean=} isPartial True for patch-like updates.
   * @return {Object}
   */
  normalizeRecord(tableName, record, isPartial) {
    var schema = this.get(tableName);
    var out = Util.clone(record || {});
    schema.columns.forEach(function(column) {
      var fieldRules = schema.fields[column] || {};
      var hasValue = !Util.isNil(out[column]);

      if (!hasValue && fieldRules.hasOwnProperty('default')) {
        var defaultValue = fieldRules.default;
        out[column] = typeof defaultValue === 'function' ? defaultValue() : defaultValue;
      }

      if (!isPartial && fieldRules.required && Util.isNil(out[column])) {
        throw new ValidationError('Field "' + column + '" is required.');
      }

      if (!Util.isNil(out[column]) && fieldRules.type) {
        var expected = fieldRules.type;
        var actual = Object.prototype.toString.call(out[column]).slice(8, -1).toLowerCase();
        if (actual !== expected.toLowerCase()) {
          throw new ValidationError(
            'Field "' + column + '" expected type ' + expected + ' but got ' + actual + '.'
          );
        }
      }
    });

    return out;
  }
}
```

FILE PATH: sheet_db_lib/06_SheetsGateway.gs

```
/**
 * Thin wrapper around SpreadsheetApp APIs.
 */
class SheetsGateway {
  /**
   * @param {Config} config
   */
  constructor(config) {
    this._config = config;
  }

  /**
   * @return {GoogleAppsScript.Spreadsheet.Spreadsheet}
   */
  getSpreadsheet() {
    if (this._config.spreadsheetId) {
      return SpreadsheetApp.openById(this._config.spreadsheetId);
    }
    return SpreadsheetApp.getActiveSpreadsheet();
  }

  /**
   * @param {string} tableName
   * @param {boolean=} createWhenMissing
   * @return {GoogleAppsScript.Spreadsheet.Sheet}
   */
  getSheet(tableName, createWhenMissing) {
    var ss = this.getSpreadsheet();
    var sheet = ss.getSheetByName(tableName);
    if (!sheet && createWhenMissing) {
      sheet = ss.insertSheet(tableName);
    }
    if (!sheet) {
      throw new StorageError('Sheet not found for table: ' + tableName);
    }
    return sheet;
  }

  /**
   * @param {string} tableName
   * @return {{headers: string[], rows: Array<Array<*>>}}
   */
  readTable(tableName) {
    var sheet = this.getSheet(tableName, false);
    var range = sheet.getDataRange();
    var values = range.getValues();
    if (values.length === 0) {
      return { headers: [], rows: [] };
    }
    return {
      headers: values[0],
      rows: values.slice(1)
    };
  }

  /**
   * Ensures table exists with given headers.
   * @param {string} tableName
   * @param {string[]} headers
   */
  ensureTable(tableName, headers) {
    var sheet = this.getSheet(tableName, true);
```

```
    var lastRow = sheet.getLastRow();
    if (lastRow === 0) {
      sheet.getRange(1, 1, 1, headers.length).setValues([headers]);
      return;
    }

    var existingHeaders = sheet.getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0]
      .map(function(value) { return String(value); });
    var expectedHeaders = headers.map(function(value) { return String(value); });
    if (existingHeaders.join('|') !== expectedHeaders.join('|')) {
      throw new StorageError('Header mismatch for table: ' + tableName);
    }
  }

  /**
   * Writes all rows for a table, replacing existing body rows.
   * @param {string} tableName
   * @param {string[]} headers
   * @param {Array<Array<*>>} rows
   */
  writeAllRows(tableName, headers, rows) {
    var sheet = this.getSheet(tableName, true);
    this.ensureTable(tableName, headers);

    var maxRows = Math.max(sheet.getMaxRows(), rows.length + 1);
    if (sheet.getMaxRows() < maxRows) {
      sheet.insertRowsAfter(sheet.getMaxRows(), maxRows - sheet.getMaxRows());
    }

    if (sheet.getLastRow() > 1) {
      sheet.getRange(2, 1, sheet.getLastRow() - 1, headers.length).clearContent();
    }

    if (rows.length > 0) {
      sheet.getRange(2, 1, rows.length, headers.length).setValues(rows);
    }
  }
}
```

FILE PATH: sheet_db_lib/07_TableRepository.gs

```
/**
 * CRUD repository for a specific table.
 */
class TableRepository {
  /**
   * @param {string} tableName
   * @param {Config} config
   * @param {SchemaRegistry} schemaRegistry
   * @param {SheetsGateway} sheetsGateway
   * @param {AuditLogger} auditLogger
   */
  constructor(tableName, config, schemaRegistry, sheetsGateway, auditLogger) {
    this._tableName = tableName;
    this._config = config;
    this._schemas = schemaRegistry;
    this._sheets = sheetsGateway;
    this._audit = auditLogger;
  }

  /**
   * @return {Array<Object>}
   */
  findAll() {
    var schema = this._schemas.get(this._tableName);
    this._sheets.ensureTable(this._tableName, schema.columns);
    var data = this._sheets.readTable(this._tableName);
    return data.rows
      .map(function(row) { return Util.rowToObject(data.headers, row); })
      .filter(function(record) { return Util.isBlank(record[this._config.deletedAtColumn]); }, thi
s);
  }

  /**
   * @param {string} id
   * @return {Object|null}
   */
  findById(id) {
    var rows = this.findAll();
    for (var i = 0; i < rows.length; i++) {
      if (rows[i][this._config.idColumn] === id) {
        return rows[i];
      }
    }
    return null;
  }

  /**
   * @param {Object} payload
   * @return {Object}
   */
  insert(payload) {
    var schema = this._schemas.get(this._tableName);
    this._sheets.ensureTable(this._tableName, schema.columns);
    var record = this._schemas.normalizeRecord(this._tableName, payload, false);
    var now = Util.nowIso();
    record[this._config.idColumn] = record[this._config.idColumn] || Util.generateId(this._tableNa
me);
    record[this._config.createdAtColumn] = record[this._config.createdAtColumn] || now;
    record[this._config.updatedAtColumn] = now;
    if (!record.hasOwnProperty(this._config.deletedAtColumn)) {
      record[this._config.deletedAtColumn] = '';
    }
```

```
    var tableData = this._sheets.readTable(this._tableName);
    var existingRows = tableData.rows.map(function(row) { return Util.rowToObject(tableData.header
s, row); });
    var idColumn = this._config.idColumn;
    var duplicate = existingRows.some(function(row) {
      return row[idColumn] === record[idColumn];
    });
    if (duplicate) {
      throw new ValidationError('Record ID already exists: ' + record[idColumn]);
    }
    existingRows.push(record);
    var rowValues = existingRows.map(function(r) { return Util.objectToRow(schema.columns, r); });
    this._sheets.writeAllRows(this._tableName, schema.columns, rowValues);
    this._audit.log('insert', this._tableName, { id: record[this._config.idColumn] });
    return record;
  }

  /**
   * @param {string} id
   * @param {Object} patch
   * @return {Object}
   */
  update(id, patch) {
    var schema = this._schemas.get(this._tableName);
    var normalizedPatch = this._schemas.normalizeRecord(this._tableName, patch, true);
    var tableData = this._sheets.readTable(this._tableName);
    var records = tableData.rows.map(function(row) { return Util.rowToObject(tableData.headers, ro
w); });
    var found = null;

    for (var i = 0; i < records.length; i++) {
      if (records[i][this._config.idColumn] === id) {
        found = records[i];
        Object.keys(normalizedPatch).forEach(function(key) {
          if (key !== this._config.idColumn && key !== this._config.createdAtColumn) {
            found[key] = normalizedPatch[key];
          }
        }, this);
        found[this._config.updatedAtColumn] = Util.nowIso();
        break;
      }
    }

    if (!found) {
      throw new StorageError('Record not found for id: ' + id);
    }

    var rowValues = records.map(function(r) { return Util.objectToRow(schema.columns, r); });
    this._sheets.writeAllRows(this._tableName, schema.columns, rowValues);
    this._audit.log('update', this._tableName, { id: id, fields: Object.keys(normalizedPatch) });
    return found;
  }

  /**
   * Soft-deletes a record.
   * @param {string} id
   * @return {Object}
   */
  remove(id) {
    var result = this.update(id, (function(col, now) {
      var patch = {};
      patch[col] = now;
      return patch;
```

```
    })(this._config.deletedAtColumn, Util.nowIso()));
    this._audit.log('remove', this._tableName, { id: id });
    return result;
  }
}
```

FILE PATH: sheet_db_lib/08_TransactionManager.gs

```
/**
 * Runs operations under ScriptLock for coarse transaction semantics.
 */
class TransactionManager {
  /**
   * @param {number=} lockTimeoutMs
   */
  constructor(lockTimeoutMs) {
    this._lockTimeoutMs = lockTimeoutMs || 30000;
  }

  /**
   * Executes callback while holding a script lock.
   * @template T
   * @param {function(): T} callback
   * @return {T}
   */
  run(callback) {
    var lock = LockService.getScriptLock();
    if (!lock.tryLock(this._lockTimeoutMs)) {
      throw new TransactionError('Failed to acquire script lock within timeout.');
    }
    try {
      return callback();
    } finally {
      lock.releaseLock();
    }
  }
}
```

FILE PATH: sheet_db_lib/09_AuditLogger.gs

```
/**
 * Minimal pluggable audit logger.
 */
class AuditLogger {
  /**
   * @param {Object=} options
   * @param {Function=} options.sink Optional custom sink function.
   */
  constructor(options) {
    var opts = options || {};
    this._sink = opts.sink || function(entry) {
      Logger.log(JSON.stringify(entry));
    };
  }

  /**
   * Emits an audit entry.
   * @param {string} action
   * @param {string} tableName
   * @param {Object=} metadata
   */
  log(action, tableName, metadata) {
    this._sink({
      at: Util.nowIso(),
      action: action,
      table: tableName,
      metadata: Util.clone(metadata || {})
    });
  }
}
```

FILE PATH: sheet_db_lib/10_ScriptProperties.gs

```
/**
 * Script Properties helper for secure and environment-specific configuration.
 */
class ScriptPropertiesHelper {
  /**
   * Creates a helper bound to Script Properties store.
   */
  constructor() {
    this._props = PropertiesService.getScriptProperties();
  }

  /**
   * Returns a property value, or a default when missing.
   * @param {string} key
   * @param {*=} defaultValue
   * @return {*}
   */
  get(key, defaultValue) {
    var value = this._props.getProperty(key);
    return value === null ? defaultValue : value;
  }

  /**
   * Returns a required property value or throws when missing/blank.
   * @param {string} key
   * @return {string}
   */
  getRequired(key) {
    var value = this.get(key, '');
    if (value === '') {
      throw new ConfigError('Missing required script property: ' + key);
    }
    return String(value);
  }

  /**
   * Returns a boolean property value.
   * Accepted true values: true, 1, yes, y, on.
   * Accepted false values: false, 0, no, n, off.
   * @param {string} key
   * @param {boolean=} defaultValue
   * @return {boolean}
   */
  getBoolean(key, defaultValue) {
    var raw = this.get(key, null);
    if (raw === null || raw === '') {
      return defaultValue === undefined ? false : !!defaultValue;
    }
    var normalized = String(raw).trim().toLowerCase();
    if (['true', '1', 'yes', 'y', 'on'].indexOf(normalized) !== -1) {
      return true;
    }
    if (['false', '0', 'no', 'n', 'off'].indexOf(normalized) !== -1) {
      return false;
    }
    throw new ConfigError('Script property is not a valid boolean: ' + key);
  }

  /**
   * Returns a numeric property value.
   * @param {string} key
   * @param {number=} defaultValue
```

```
 * @return {number}
 */
getNumber(key, defaultValue) {
  var raw = this.get(key, null);
  if (raw === null || raw === '') {
    return defaultValue === undefined ? 0 : Number(defaultValue);
  }
  var n = Number(raw);
  if (isNaN(n)) {
    throw new ConfigError('Script property is not a valid number: ' + key);
  }
  return n;
}

/**
 * Returns all script properties as key-value object.
 * @return {Object<string, string>}
 */
getAll() {
  return this._props.getProperties();
}

/**
 * Returns all script properties with sensitive values redacted.
 * @return {Object<string, string>}
 */
getAllRedacted() {
  var all = this.getAll();
  var out = {};
  Object.keys(all).forEach(function(key) {
    out[key] = ScriptPropertiesHelper.shouldRedactKey(key) ? '[REDACTED]' : all[key];
  });
  return out;
}

/**
 * Sets a script property value.
 * @param {string} key
 * @param {*} value
 */
set(key, value) {
  this._props.setProperty(key, String(value));
}

/**
 * Sets multiple script properties.
 * @param {Object<string, *>} obj
 */
setMany(obj) {
  var stringified = {};
  Object.keys(obj || {}).forEach(function(key) {
    stringified[key] = String(obj[key]);
  });
  this._props.setProperties(stringified, false);
}

/**
 * Deletes a script property.
 * @param {string} key
 */
delete(key) {
  this._props.deleteProperty(key);
}
```

```
  /**
   * Returns true when a script property exists and is non-empty.
   * @param {string} key
   * @return {boolean}
   */
  has(key) {
    var value = this._props.getProperty(key);
    return value !== null && value !== '';
  }

  /**
   * Returns true when key should be redacted in debug output.
   * @param {string} key
   * @return {boolean}
   */
  static shouldRedactKey(key) {
    var upper = String(key || '').toUpperCase();
    if (upper === 'APP_PASSCODE' || upper === 'SIGNING_SECRET' || upper === 'WEBHOOK_SECRET' || up
per === 'API_KEY') {
      return true;
    }
    return upper.indexOf('TOKEN') !== -1 || upper.indexOf('SECRET') !== -1 || upper.indexOf('PASSW
ORD') !== -1;
  }
}

/**
 * Reads SheetDb client-related config from Script Properties.
 *
 * Supported keys:
 * - SHEET_DB_SPREADSHEET_ID
 * - SHEET_DB_ID_COLUMN
 * - SHEET_DB_CREATED_AT_COLUMN
 * - SHEET_DB_UPDATED_AT_COLUMN
 * - SHEET_DB_DELETED_AT_COLUMN
 * - SHEET_DB_LOCK_TIMEOUT_MS
 *
 * Also reserved for host-layer secure use:
 * - APP_PASSCODE
 * - SIGNING_SECRET
 * - WEBHOOK_SECRET
 * - API_KEY
 *
 * @return {Object}
 */
function readSheetDbConfigFromScriptProperties() {
  var helper = new ScriptPropertiesHelper();
  var config = {};

  if (helper.has('SHEET_DB_SPREADSHEET_ID')) {
    config.spreadsheetId = helper.get('SHEET_DB_SPREADSHEET_ID', '');
  }
  if (helper.has('SHEET_DB_ID_COLUMN')) {
    config.idColumn = helper.get('SHEET_DB_ID_COLUMN', 'id');
  }
  if (helper.has('SHEET_DB_CREATED_AT_COLUMN')) {
    config.createdAtColumn = helper.get('SHEET_DB_CREATED_AT_COLUMN', 'createdAt');
  }
  if (helper.has('SHEET_DB_UPDATED_AT_COLUMN')) {
    config.updatedAtColumn = helper.get('SHEET_DB_UPDATED_AT_COLUMN', 'updatedAt');
  }
  if (helper.has('SHEET_DB_DELETED_AT_COLUMN')) {
    config.deletedAtColumn = helper.get('SHEET_DB_DELETED_AT_COLUMN', 'deletedAt');
  }
```

```
  if (helper.has('SHEET_DB_LOCK_TIMEOUT_MS')) {
    config.lockTimeoutMs = helper.getNumber('SHEET_DB_LOCK_TIMEOUT_MS', 30000);
  }

  return config;
}

/**
 * Merges explicit client options with script properties-based config.
 * Precedence: explicit options > script properties > hardcoded defaults.
 *
 * @param {Object=} options
 * @return {Object}
 */
function mergeClientOptionsWithScriptProperties(options) {
  var fromProps = readSheetDbConfigFromScriptProperties();
  var explicit = options || {};
  var merged = {};
  Object.keys(fromProps).forEach(function(key) {
    merged[key] = fromProps[key];
  });
  Object.keys(explicit).forEach(function(key) {
    merged[key] = explicit[key];
  });
  return merged;
}

/**
 * Example setup helper for consumers (manual one-time setup).
 *
 * Example:
 * PropertiesService.getScriptProperties().setProperty('SHEET_DB_SPREADSHEET_ID', '...');
 */
function exampleSetSheetDbScriptProperties() {
  // Intentionally no-op; serves as in-library usage documentation only.
}
```

FILE PATH: slack_host/00_WebAppEntry.gs

```
/**
 * Slack host entrypoint (web app).
 */

/**
 * Creates a DB client from SheetDb library and registers host schemas.
 * @return {DbClient}
 */
function createHostDbClient() {
  var db = SheetDb.createClientFromScriptProperties();

  db.schema('students', {
    columns: ['id', 'name', 'source', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      id: { required: true, type: 'string' },
      name: { required: true, type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('enrollments', {
    columns: ['id', 'studentId', 'courseId', 'status', 'source', 'createdAt', 'updatedAt', 'delete
dAt'],
    fields: {
      studentId: { required: true, type: 'string' },
      courseId: { required: true, type: 'string' },
      status: { default: 'enrolled', type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('automations', {
    columns: ['id', 'requestId', 'workflow', 'eventType', 'state', 'createdAt', 'updatedAt', 'dele
tedAt'],
    fields: {
      requestId: { required: true, type: 'string' },
      workflow: { required: true, type: 'string' },
      eventType: { required: true, type: 'string' },
      state: { default: 'received', type: 'string' }
    }
  });

  db.schema('audit_logs', {
    columns: ['id', 'requestId', 'source', 'action', 'status', 'message', 'metadata', 'createdAt',
 'updatedAt', 'deletedAt'],
    fields: {
      requestId: { required: true, type: 'string' },
      source: { required: true, type: 'string' },
      action: { required: true, type: 'string' },
      status: { required: true, type: 'string' },
      message: { type: 'string' },
      metadata: { type: 'string' }
    }
  });

  return db;
}

/**
 * Creates the host-layer router with all dependencies wired.
 * @param {DbClient} db
 * @param {HostConfig} hostConfig
```

```
 * @return {SlackRouter}
 */
function createHostRouter(db, hostConfig) {
  var parser = new SlackPayloadParser();
  var security = new SlackSecurity(hostConfig);
  var contexts = new RequestContextFactory(hostConfig);
  var formatter = new ResultFormatter();

  var enrollmentService = new LmsEnrollmentService(db);
  var automationService = new LmsAutomationService(db);
  var workflowHandlers = new WorkflowWebhookHandlers(enrollmentService, automationService);
  var slackService = new SlackService(enrollmentService, workflowHandlers, formatter);

  return new SlackRouter(parser, security, contexts, slackService, formatter, db);
}

/**
 * Webhook entrypoint.
 * @param {GoogleAppsScript.Events.DoPost} e
 * @return {GoogleAppsScript.Content.TextOutput}
 */
function doPost(e) {
  var db = createHostDbClient();
  var hostConfig = new HostConfig(new ScriptPropertiesHelper());
  var router = createHostRouter(db, hostConfig);
  var result = router.handle(e);

  return ContentService
    .createTextOutput(JSON.stringify(result.slack || result))
    .setMimeType(ContentService.MimeType.JSON);
}
```

FILE PATH: slack_host/01_SlackRouter.gs

```
/**
 * HTTP router boundary: parse -> verify -> context -> app service.
 */
class SlackRouter {
  /**
   * @param {SlackPayloadParser} parser
   * @param {SlackSecurity} security
   * @param {RequestContextFactory} contextFactory
   * @param {SlackService} slackService
   * @param {ResultFormatter} resultFormatter
   * @param {DbClient} db
   */
  constructor(parser, security, contextFactory, slackService, resultFormatter, db) {
    this._parser = parser;
    this._security = security;
    this._contexts = contextFactory;
    this._service = slackService;
    this._results = resultFormatter;
    this._db = db;
  }

  /**
   * @param {GoogleAppsScript.Events.DoPost} e
   * @return {Object}
   */
  handle(e) {
    var parsed = this._parser.parseEvent(e);
    var context = this._contexts.create(parsed.routeType, parsed.payload, parsed.envelope);

    var auth = this._security.verify(parsed);
    if (!auth.ok) {
      var denied = this._results.failure(auth.code, auth.message, context, {});
      this._writeAudit(context, denied);
      return denied;
    }

    var result;
    try {
      if (parsed.routeType === 'slash_command') {
        result = this._service.handleSlashCommand(context, parsed.payload);
      } else if (parsed.routeType === 'workflow_webhook') {
        result = this._service.handleWorkflowWebhook(context, parsed.payload);
      } else {
        result = this._results.failure('UNSUPPORTED_PAYLOAD', 'Unsupported payload.', context, {})
;
      }
    } catch (err) {
      result = this._results.failure(err.code || 'HOST_ERROR', err.message || 'Unexpected error.',
 context, {});
    }

    this._writeAudit(context, result);
    return result;
  }

  /** @private */
  _writeAudit(context, result) {
    this._db.table('audit_logs').insert({
      requestId: context.requestId,
      source: context.source,
      action: context.routeType,
      status: result.ok ? 'success' : 'failure',
```

```
      message: result.message,
      metadata: JSON.stringify({ code: result.code, requestId: result.requestId })
    });
  }
}
```

FILE PATH: slack_host/02_SlackPayloadParser.gs

```
/**
 * Parses and normalizes inbound Slack-related payloads.
 */
class SlackPayloadParser {
  /**
   * @param {GoogleAppsScript.Events.DoPost} e
   * @return {{routeType: string, payload: Object, envelope: Object}}
   */
  parseEvent(e) {
    var rawBody = (e && e.postData && e.postData.contents) ? e.postData.contents : '';
    var params = this._parseFormEncoded(rawBody);

    if (params.command) {
      return {
        routeType: 'slash_command',
        payload: this._normalizeSlashCommand(params),
        envelope: { rawBody: rawBody, params: params }
      };
    }

    var jsonPayload = this._parseJson(rawBody);
    if (jsonPayload) {
      return {
        routeType: 'workflow_webhook',
        payload: this._normalizeWorkflowWebhook(jsonPayload),
        envelope: { rawBody: rawBody, json: jsonPayload }
      };
    }

    return { routeType: 'unknown', payload: {}, envelope: { rawBody: rawBody } };
  }

  /** @private */
  _normalizeSlashCommand(payload) {
    return {
      command: String(payload.command || '').trim(),
      text: String(payload.text || '').trim(),
      teamId: String(payload.team_id || ''),
      channelId: String(payload.channel_id || ''),
      userId: String(payload.user_id || ''),
      triggerId: String(payload.trigger_id || ''),
      responseUrl: String(payload.response_url || ''),
      requestId: String(payload.request_id || Util.generateId('slash'))
    };
  }

  /** @private */
  _normalizeWorkflowWebhook(payload) {
    var data = payload.data || payload;
    return {
      workflow: String(payload.workflow || payload.workflow_name || 'unknown_workflow'),
      eventType: String(payload.eventType || payload.event_type || 'workflow_event'),
      requestId: String(payload.requestId || payload.request_id || Util.generateId('wf')),
      studentId: String(data.studentId || data.student_id || ''),
      courseId: String(data.courseId || data.course_id || ''),
      actorId: String(data.actorId || data.actor_id || ''),
      status: String(data.status || '')
    };
  }

  /** @private */
  _parseJson(raw) {
```

```
    try {
      if (!raw) return null;
      return JSON.parse(raw);
    } catch (err) {
      return null;
    }
  }

  /** @private */
  _parseFormEncoded(body) {
    var out = {};
    if (!body) return out;
    body.split('&').forEach(function(pair) {
      var tokens = pair.split('=');
      var key = decodeURIComponent(tokens[0] || '').replace(/\+/g, ' ');
      var rawValue = tokens.length > 1 ? tokens.slice(1).join('=') : '';
      var value = decodeURIComponent(rawValue).replace(/\+/g, ' ');
      if (key) out[key] = value;
    });
    return out;
  }
}
```

FILE PATH: slack_host/03_SlackService.gs

```
/**
 * Application service that orchestrates Slack routes into LMS services.
 */
class SlackService {
  /**
   * @param {LmsEnrollmentService} enrollmentService
   * @param {WorkflowWebhookHandlers} workflowHandlers
   * @param {ResultFormatter} resultFormatter
   */
  constructor(enrollmentService, workflowHandlers, resultFormatter) {
    this._enrollment = enrollmentService;
    this._workflow = workflowHandlers;
    this._results = resultFormatter;
  }

  /**
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  handleSlashCommand(context, payload) {
    if (payload.command === '/enroll-student') {
      var result = this._enrollment.enrollFromSlash(context, payload);
      return result.ok ? this._results.success(result.message, result.data, context)
        : this._results.failure(result.code, result.message, context, result.data);
    }

    return this._results.failure('UNSUPPORTED_COMMAND', 'Unsupported command: ' + payload.command,
 context, {});
  }

  /**
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  handleWorkflowWebhook(context, payload) {
    var result = this._workflow.handle(context, payload);
    return result.ok ? this._results.success(result.message, result.data, context)
      : this._results.failure(result.code, result.message, context, result.data);
  }
}
```

FILE PATH: slack_host/04_SlackSecurity.gs

```
/**
 * Host-layer security checks for Slack/webhook calls.
 */
class SlackSecurity {
  /**
   * @param {HostConfig} hostConfig
   */
  constructor(hostConfig) {
    this._config = hostConfig;
  }

  /**
   * Best-effort request verification for Apps Script web-app requests.
   * Uses passcode from query/form when available.
   *
   * @param {{routeType: string, payload: Object, envelope: Object}} parsed
   * @return {{ok: boolean, code: string, message: string}}
   */
  verify(parsed) {
    var configured = this._config.getAppPasscode();
    if (!configured) {
      return { ok: true, code: 'NO_PASSCODE_CONFIGURED', message: 'Passcode check skipped.' };
    }

    var params = (parsed.envelope && parsed.envelope.params) ? parsed.envelope.params : {};
    var supplied = String(params.passcode || params.api_key || '');
    if (supplied && supplied === configured) {
      return { ok: true, code: 'AUTHORIZED', message: 'Passcode validated.' };
    }

    return { ok: false, code: 'UNAUTHORIZED', message: 'Invalid or missing passcode.' };
  }
}
```

FILE PATH: slack_host/05_LmsEnrollmentService.gs

```
/**
 * LMS enrollment operations implemented using the SheetDb library only.
 */
class LmsEnrollmentService {
  /**
   * @param {DbClient} db
   */
  constructor(db) {
    this._db = db;
  }

  /**
   * Slash-command flow example:
   * /enroll-student <studentId> <courseId>
   *
   * Upserts into ‘students‘ and ‘enrollments‘, then writes audit log row.
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  enrollFromSlash(context, payload) {
    var args = String(payload.text || ’’).split(/\s+/).filter(function(v) { return !!v; });
    if (args.length < 2) {
      return { ok: false, code: ’INVALID_ARGUMENTS’, message: ’Usage: /enroll-student <studentId>
<courseId>’, data: {} };
    }

    var studentId = args[0];
    var courseId = args[1];
    var out = this._upsertEnrollment(context, studentId, courseId, ’slash_command’);
    return out;
  }

  /**
   * Workflow webhook flow example for enrollment updates/inserts.
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  upsertFromWorkflow(context, payload) {
    if (!payload.studentId || !payload.courseId) {
      return { ok: false, code: ’INVALID_WORKFLOW_PAYLOAD’, message: ’studentId and courseId are r
equired.’, data: {} };
    }

    return this._upsertEnrollment(context, payload.studentId, payload.courseId, ’workflow_webhook’
, payload.status || ’enrolled’);
  }

  /** @private */
  _upsertEnrollment(context, studentId, courseId, source, status) {
    var students = this._db.table(’students’);
    var enrollments = this._db.table(’enrollments’);
    var audit = this._db.table(’audit_logs’);
    var finalStatus = status || ’enrolled’;
    var enrollmentRecord;

    this._db.transaction(function() {
      var existingStudent = students.findById(studentId);
      if (!existingStudent) {
        students.insert({ id: studentId, name: studentId, source: source });
      }
```

```
      var existingEnrollment = enrollments.findAll().filter(function(r) {
        return r.studentId === studentId && r.courseId === courseId;
      })[0];

      if (existingEnrollment) {
        enrollmentRecord = enrollments.update(existingEnrollment.id, {
          status: finalStatus,
          source: source
        });
      } else {
        enrollmentRecord = enrollments.insert({
          studentId: studentId,
          courseId: courseId,
          status: finalStatus,
          source: source
        });
      }

      audit.insert({
        requestId: context.requestId,
        source: context.source,
        action: 'enrollment_upsert',
        status: 'success',
        message: 'Enrollment upsert completed.',
        metadata: JSON.stringify({ studentId: studentId, courseId: courseId, enrollmentId: enrollm
entRecord.id })
      });
    });

    this._db.audit('enrollment_upsert', 'enrollments', {
      requestId: context.requestId,
      studentId: studentId,
      courseId: courseId,
      enrollmentId: enrollmentRecord.id
    });

    return {
      ok: true,
      code: 'ENROLLMENT_UPSERTED',
      message: 'Enrollment upserted successfully.',
      data: {
        enrollmentId: enrollmentRecord.id,
        studentId: studentId,
        courseId: courseId,
        status: enrollmentRecord.status
      }
    };
  }
}
```

FILE PATH: slack_host/06_LmsAutomationService.gs

```
/**
 * LMS automation operations that persist workflow state via SheetDb.
 */
class LmsAutomationService {
  /**
   * @param {DbClient} db
   */
  constructor(db) {
    this._db = db;
  }

  /**
   * Inserts/updates an automation row and writes an audit log row.
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  upsertAutomation(context, payload) {
    var automations = this._db.table('automations');
    var audit = this._db.table('audit_logs');
    var workflow = payload.workflow || 'unknown_workflow';
    var eventType = payload.eventType || 'workflow_event';
    var state = payload.status || 'received';

    var existing = automations.findAll().filter(function(r) {
      return r.workflow === workflow && r.requestId === payload.requestId;
    })[0];

    var row;
    if (existing) {
      row = automations.update(existing.id, { eventType: eventType, state: state });
    } else {
      row = automations.insert({ requestId: payload.requestId, workflow: workflow, eventType: even
tType, state: state });
    }

    audit.insert({
      requestId: context.requestId,
      source: context.source,
      action: 'automation_upsert',
      status: 'success',
      message: 'Automation row upserted.',
      metadata: JSON.stringify({ automationId: row.id, workflow: workflow, eventType: eventType })
    });

    return {
      ok: true,
      code: 'AUTOMATION_UPSERTED',
      message: 'Automation row upserted.',
      data: { automationId: row.id, workflow: workflow, eventType: eventType, state: row.state }
    };
  }
}
```

FILE PATH: slack_host/07_RequestContextFactory.gs

```
/**
 * Creates normalized request-context objects for host handlers.
 */
class RequestContextFactory {
  /**
   * @param {HostConfig} hostConfig
   */
  constructor(hostConfig) {
    this._config = hostConfig;
  }

  /**
   * @param {string} routeType
   * @param {Object} payload
   * @param {Object=} envelope
   * @return {Object}
   */
  create(routeType, payload, envelope) {
    var env = envelope || {};
    return {
      requestId: payload.requestId || Util.generateId('req'),
      receivedAt: Util.nowIso(),
      routeType: routeType,
      source: routeType,
      actorId: payload.userId || payload.actorId || 'unknown',
      teamId: payload.teamId || '',
      channelId: payload.channelId || this._config.getSlackDefaultChannel(),
      raw: env.rawBody || ''
    };
  }
}
```

FILE PATH: slack_host/08_ResultFormatter.gs

```
/**
 * Formats host operation results into structured Slack-friendly objects.
 */
class ResultFormatter {
  /**
   * @param {string} message
   * @param {Object=} data
   * @param {Object=} context
   * @return {Object}
   */
  success(message, data, context) {
    return {
      ok: true,
      code: 'OK',
      message: message,
      requestId: (context && context.requestId) || '',
      data: data || {},
      slack: {
        response_type: 'ephemeral',
        text: message
      }
    };
  }

  /**
   * @param {string} code
   * @param {string} message
   * @param {Object=} context
   * @param {Object=} data
   * @return {Object}
   */
  failure(code, message, context, data) {
    return {
      ok: false,
      code: code || 'ERROR',
      message: message || 'Unknown error.',
      requestId: (context && context.requestId) || '',
      data: data || {},
      slack: {
        response_type: 'ephemeral',
        text: ':warning: ' + (message || 'Unknown error.')
      }
    };
  }
}
```

FILE PATH: slack_host/09_WorkflowWebhookHandlers.gs

```
/**
 * Handles normalized workflow webhook events.
 */
class WorkflowWebhookHandlers {
  /**
   * @param {LmsEnrollmentService} enrollmentService
   * @param {LmsAutomationService} automationService
   */
  constructor(enrollmentService, automationService) {
    this._enrollment = enrollmentService;
    this._automation = automationService;
  }

  /**
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  handle(context, payload) {
    if (payload.eventType === 'enrollment_requested') {
      return this._enrollment.upsertFromWorkflow(context, payload);
    }
    return this._automation.upsertAutomation(context, payload);
  }
}
```

FILE PATH: slack_host/10_HostConfig.gs

```
/**
 * Host-layer configuration backed by Script Properties.
 */
class HostConfig {
  /**
   * @param {ScriptPropertiesHelper=} helper
   */
  constructor(helper) {
    this._props = helper || new ScriptPropertiesHelper();
  }

  /** @return {string} */
  getAppPasscode() { return this._props.get('APP_PASSCODE', ''); }
  /** @return {string} */
  getSigningSecret() { return this._props.get('SIGNING_SECRET', ''); }
  /** @return {string} */
  getWebhookSecret() { return this._props.get('WEBHOOK_SECRET', ''); }
  /** @return {string} */
  getApiKey() { return this._props.get('API_KEY', ''); }
  /** @return {string} */
  getSlackBotToken() { return this._props.get('SLACK_BOT_TOKEN', ''); }
  /** @return {string} */
  getSlackSigningSecret() { return this._props.get('SLACK_SIGNING_SECRET', this.getSigningSecret()
); }
  /** @return {string} */
  getSlackDefaultChannel() { return this._props.get('SLACK_DEFAULT_CHANNEL', ''); }

  /**
   * @return {Object}
   */
  getClientOverrides() {
    return mergeClientOptionsWithScriptProperties({});
  }
}
```

FILE PATH: slack_host/11_HostTestHarness.gs

```
/**
 * Manual Apps Script test harness for the split Slack host architecture.
 */

/**
 * Builds host dependencies for manual tests.
 * Uses TEST_SPREADSHEET_ID script property when available.
 * @return {{db: DbClient, hostConfig: HostConfig, router: SlackRouter}}
 */
function _buildHostTestContext() {
  var props = new ScriptPropertiesHelper();
  var testSpreadsheetId = props.get('TEST_SPREADSHEET_ID', '');
  var options = testSpreadsheetId ? { spreadsheetId: testSpreadsheetId } : {};

  var db = SheetDb.createClientFromScriptProperties(options);

  db.schema('students', {
    columns: ['id', 'name', 'source', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      id: { required: true, type: 'string' },
      name: { required: true, type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('enrollments', {
    columns: ['id', 'studentId', 'courseId', 'status', 'source', 'createdAt', 'updatedAt', 'delete
dAt'],
    fields: {
      studentId: { required: true, type: 'string' },
      courseId: { required: true, type: 'string' },
      status: { default: 'enrolled', type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('automations', {
    columns: ['id', 'requestId', 'workflow', 'eventType', 'state', 'createdAt', 'updatedAt', 'dele
tedAt'],
    fields: {
      requestId: { required: true, type: 'string' },
      workflow: { required: true, type: 'string' },
      eventType: { required: true, type: 'string' },
      state: { default: 'received', type: 'string' }
    }
  });

  db.schema('audit_logs', {
    columns: ['id', 'requestId', 'source', 'action', 'status', 'message', 'metadata', 'createdAt',
 'updatedAt', 'deletedAt'],
    fields: {
      requestId: { required: true, type: 'string' },
      source: { required: true, type: 'string' },
      action: { required: true, type: 'string' },
      status: { required: true, type: 'string' },
      message: { type: 'string' },
      metadata: { type: 'string' }
    }
  });

  var hostConfig = new HostConfig(props);
  var router = createHostRouter(db, hostConfig);
```

```
  return { db: db, hostConfig: hostConfig, router: router };
}

/**
 * Encodes form payload.
 * @param {Object} obj
 * @return {string}
 */
function _toFormBody(obj) {
  return Object.keys(obj).map(function(key) {
    return encodeURIComponent(key) + '=' + encodeURIComponent(String(obj[key]));
  }).join('&');
}

/**
 * Builds a mock DoPost event object.
 * @param {string} body
 * @return {GoogleAppsScript.Events.DoPost}
 */
function _mockDoPostEvent(body) {
  return {
    postData: {
      contents: body,
      type: 'application/x-www-form-urlencoded'
    }
  };
}

/**
 * testSlashEnrollMock
 * Simulates '/enroll-student' and runs full router path.
 */
function testSlashEnrollMock() {
  var ctx = _buildHostTestContext();
  var suffix = new Date().getTime();
  var passcode = ctx.hostConfig.getAppPasscode();
  var payload = {
    command: '/enroll-student',
    text: 'student-' + suffix + ' course-' + suffix,
    user_id: 'U_TEST_' + suffix,
    team_id: 'T_TEST',
    channel_id: 'C_TEST'
  };
  if (passcode) {
    payload.passcode = passcode;
  }

  var event = _mockDoPostEvent(_toFormBody(payload));
  var result = ctx.router.handle(event);
  Logger.log('[testSlashEnrollMock] ' + JSON.stringify(result));
}

/**
 * testWorkflowEnrollmentMock
 * Simulates 'enrollment_requested' workflow webhook and runs full router path.
 */
function testWorkflowEnrollmentMock() {
  var ctx = _buildHostTestContext();
  var suffix = new Date().getTime();
  var payload = {
    workflow: 'lms_enrollment',
    eventType: 'enrollment_requested',
    requestId: 'wf_req_' + suffix,
    data: {
```

```
      studentId: 'wf-student-' + suffix,
      courseId: 'wf-course-' + suffix,
      actorId: 'WF_ACTOR'
    }
  };
  var event = _mockDoPostEvent(JSON.stringify(payload));
  var result = ctx.router.handle(event);
  Logger.log('[testWorkflowEnrollmentMock] ' + JSON.stringify(result));
}

/**
 * testAutomationMock
 * Simulates generic automation workflow webhook and runs full router path.
 */
function testAutomationMock() {
  var ctx = _buildHostTestContext();
  var suffix = new Date().getTime();
  var payload = {
    workflow: 'lms_automation',
    eventType: 'automation_ping',
    requestId: 'automation_req_' + suffix,
    data: {
      status: 'queued',
      actorId: 'AUTO_ACTOR'
    }
  };
  var event = _mockDoPostEvent(JSON.stringify(payload));
  var result = ctx.router.handle(event);
  Logger.log('[testAutomationMock] ' + JSON.stringify(result));
}

/**
 * testHostConfig
 * Verifies host config reads script properties without logging raw secrets.
 */
function testHostConfig() {
  var props = new ScriptPropertiesHelper();
  var hostConfig = new HostConfig(props);
  var keys = [
    'APP_PASSCODE',
    'SIGNING_SECRET',
    'WEBHOOK_SECRET',
    'API_KEY',
    'SLACK_BOT_TOKEN',
    'SLACK_SIGNING_SECRET',
    'SLACK_DEFAULT_CHANNEL'
  ];

  var values = {
    APP_PASSCODE: hostConfig.getAppPasscode(),
    SIGNING_SECRET: hostConfig.getSigningSecret(),
    WEBHOOK_SECRET: hostConfig.getWebhookSecret(),
    API_KEY: hostConfig.getApiKey(),
    SLACK_BOT_TOKEN: hostConfig.getSlackBotToken(),
    SLACK_SIGNING_SECRET: hostConfig.getSlackSigningSecret(),
    SLACK_DEFAULT_CHANNEL: hostConfig.getSlackDefaultChannel()
  };

  var redacted = {};
  keys.forEach(function(key) {
    var value = values[key] || '';
    redacted[key] = ScriptPropertiesHelper.shouldRedactKey(key)
      ? (value ? '[REDACTED_SET]' : '[REDACTED_EMPTY]')
      : value;
```

```
  });

  Logger.log('[testHostConfig] ' + JSON.stringify(redacted));
}

/**
 * testAuditWrite
 * Executes one mutation and verifies an audit row is written.
 */
function testAuditWrite() {
  var ctx = _buildHostTestContext();
  var auditTable = ctx.db.table('audit_logs');
  var before = auditTable.findAll().length;

  var suffix = new Date().getTime();
  var passcode = ctx.hostConfig.getAppPasscode();
  var payload = {
    command: '/enroll-student',
    text: 'audit-student-' + suffix + ' audit-course-' + suffix,
    user_id: 'U_AUDIT_' + suffix,
    team_id: 'T_AUDIT',
    channel_id: 'C_AUDIT'
  };
  if (passcode) {
    payload.passcode = passcode;
  }

  var event = _mockDoPostEvent(_toFormBody(payload));
  var result = ctx.router.handle(event);
  var after = auditTable.findAll().length;

  Logger.log('[testAuditWrite] ' + JSON.stringify({
    mutationOk: !!result.ok,
    auditRowsBefore: before,
    auditRowsAfter: after,
    auditRowDelta: after - before
  }));
}
```

Missing / Inaccessible Files Appendix
None